ReasoningFlow: Discourse Structures for Understanding LLM Reasoning Traces

Jinu Lee, Shivam Agarwal, Amruta Parulekar, Siddarth Madala,
Dilek Hakkani-Tür, Julia Hockenmaier
The Grainger College of Engineering
University of Illinois Urbana-Champaign
{jinulee2, shivam2, amp20, smadala2, dilek, juliahmr}@illinois.edu

Abstract

Large reasoning models (LRMs) produce reasoning traces with non-linear structures, such as backtracking and self-correction, that complicate the evaluation and monitoring of the reasoning process. We introduce **ReasoningFlow**, a framework that captures the discourse structures of LRM reasoning traces into fine-grained directed acyclic graphs (DAGs). We develop and validate our annotation schema through careful manual annotation of 31 traces (2.1k steps), achieving high inter-annotator agreement, then scale to automatic annotation of 1,260 traces (247.7k steps) spanning three tasks (math, science, argumentation) and five models (Qwen2.5-32B-Inst, QwQ-32B, DeepSeek-V3, DeepSeek-R1, GPT-oss-120B). By analyzing ReasoningFlow graphs, we find: (1) LRMs exhibit structurally similar traces, despite being trained from different base models and potentially non-overlapping post-training data. (2) ReasoningFlow reveals diverse fine-grained reasoning behaviors (e.g., local verification, self-reflection, and assumptions) that can be used for better reasoning trace monitorability. (3) In LRMs, most of the erroneous steps are not used to derive final answers. (4) Mechanistic causal dependencies between steps do not reflect the language-level discourse structure. We release the dataset and code in: Homepage.

1 Introduction

Large Reasoning Models (LRMs; e.g., DeepSeek-R1 (Guo et al., 2025)) generate extended reasoning traces with non-linear reasoning behaviors, such as verification, self-reflection, and backtracking (Gandhi et al., 2025). This non-linearity complicates both correctness evaluation and faithfulness monitoring. For instance, stepwise evaluation (Lightman et al., 2024) may flag an erroneous step, yet the trace as a whole may still be correct if the self-verification overrides the previous error.

Recent attempts to understand the non-linear structure of LRM traces either lack expressive relation labels or only annotate inter-paragraph structures (Bogdan et al., 2025; Jiang et al., 2025; Marjanovic et al., 2026), which are too coarse for annotating fine-grained reasoning behaviors. On the other hand, discourse structure annotations for human text (Carlson et al., 2001; Stab and Gurevych, 2017) fail to capture the relations and structures emerging in goal-oriented reasoning traces.

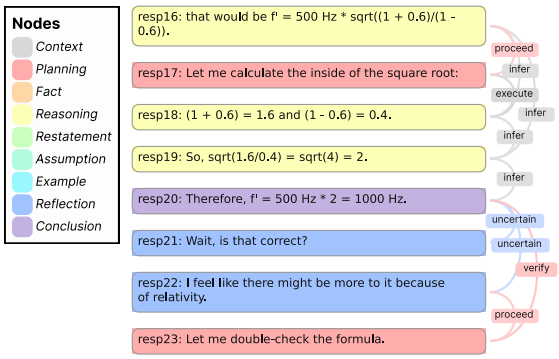


Figure 1: Example of a ReasoningFlow graph. ReasoningFlow segments the reasoning trace into **nodes**, and annotates the relations between the nodes as **edges**. This example shows deductive reasoning (`resp16-20`) and self-reflection/verification (`resp20-23`) behaviors.

We develop **ReasoningFlow**, a framework for annotating fine-grained discourse structures of reasoning traces. ReasoningFlow converts reasoning traces into a directed acyclic graph with 8 node types and 14 edge types. We release 31 manually annotated and cross-verified reasoning traces (2.1k steps), along with 1,260 automatically annotated traces (247.7k steps) generated by five models (Qwen2.5-32B-Inst, QwQ-32B, DeepSeek-V3, DeepSeek-R1, GPT-oss-120B) across math, science, and argumentation tasks.

ReasoningFlow can be used to improve the monitorability and faithfulness of reasoning traces. Using ReasoningFlow, we find:

- LRMs across different families and sizes demonstrate similar reasoning trace structures.

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 15430–15463
October 24-29, 2026 ©2026 Association for Computational Linguistics

Schema	LRM?	# Node	# Edge	Granularity	Structure	IAA?
PARC (Mukherjee et al., 2025)	X	1	1	Paragraph	**DAG**	O
Thought Anchors (Bogdan et al., 2025)	O	**8**	1	Sent.	**DAG**	O
R1-Thoughtology (Marjanovic et al., 2026)	O	4	-	Paragraph	Linear	X
LCoT2Tree (Jiang et al., 2025)	O	1	4	Paragraph	Tree	X
ReJump (Zeng et al., 2025b)	O	1	3	Paragraph	Tree	X
ReasoningFlow (this work)	O	**8**	**14**	**Sub-sent.**	**DAG**	**O**

Table 1: Comparison of structure annotation schema for LLM reasoning traces. Each column corresponds to: (LRM?) whether it is designed to accommodate LRMs' long traces, (# Node) number of node labels, (# Edge) number of edge labels, (Granularity) granularity of nodes, (Structure) the generated graph structure, and (IAA?) whether multiple human annotators verified the schema. At the time of writing, ReasoningFlow is the only work to annotate fine-grained nodes and edges, and validate with inter-annotator agreement analysis.

- ReasoningFlow can identify fine-grained reasoning behaviors like *local verification, self-reflection, and assumptions*, enabling a new dimension for monitoring reasoning traces.
- Most erroneous steps in LRMs do not lead to incorrect final answers, explaining why error detection does not directly transfer to better performance in LRMs.
- Mechanistically measured step-to-step causal dependencies (Bogdan et al., 2025) do not reflect text-level discourse relations.

2 Related Works

2.1 Reasoning trace structures

Prior to LRMs, reasoning traces were commonly viewed as entailment graphs, which anchor each step to its logical premises (Ling et al., 2023; Mukherjee et al., 2025). Yet, these graphs only show logical entailments; therefore, they cannot capture the diverse reasoning patterns exhibited by LRMs, including planning and verification.

Early efforts to analyze LRM traces focus on modeling verification. Marjanovic et al. (2026) treats an LRM trace as an initial solution followed by iterative verification attempts, while Jiang et al. (2025); Zeng et al. (2025b) models traces as *trees of paragraphs* (Yao et al., 2023) with verification and backtracking hyperedges. However, both approaches are too coarse to characterize the full spectrum of reasoning behaviors, which can occur down to the sub-sentence level (Section 6).

A more fundamental limitation cuts across all of this work: none validates its annotation scheme inter-annotator agreement, leaving open the question of whether the proposed frameworks are consistently interpretable. Given the linguistic complexity of LRM reasoning traces, there is a strong need for a framework that is both expressive enough to capture diverse reasoning patterns and reliable enough to be applied consistently by human annotators. Table 1 compares related works on LLM reasoning trace structure annotation.

2.2 Discourse/Argumentation structures

Discourse and argumentation parsing frameworks have been widely applied to capture the semantic structures of long texts.

Discourse structures. Rhetorical Structure Theory (RST) builds hierarchical discourse structures with directed relations between two clauses (Mann and Thompson, 1988). RST defines over 20 relation types between clauses (e.g., elaboration, cause, condition), covering diverse rhetorical intent of the author (Carlson et al., 2001).

Argumentation structures capture the argumentational role of a text span (major claim, minor claim, premise) and the semantic relation between two spans (*Does this span support or attack the corresponding claim?*) (Stab and Gurevych, 2017). The resulting document structure is typically represented as a tree, in which atomic premises connect recursively up to a major claim.

However, both approaches are not fully compatible with LLM reasoning traces for two reasons. First, existing schemas are not designed to accommodate reasoning trace-specific phenomena like *verification*, which rarely appear in well-organized news or argumentative texts. Second, autoregressively generated LRM traces exhibit only left-to-right causal dependencies as in improvised human speech (Kempen and Hoenkamp, 1987), where organized texts frequently exhibit *backward* dependencies. Together, these underscore the need for an annotation schema designed specifically for reasoning traces.

Label	Definition
Node labels	
`Context`	User-provided text: problem statement, retrieved documents, tool responses.
`Planning`	States what to do next; sets a global direction or a local goal for the following nodes.
`Fact`	Parametric knowledge (laws, definitions, constants) independent of the context and of prior nodes.
`Reasoning`	Deductive, inductive, or abductive inference that derives new content from previous nodes.
`Restatement`	Copies or paraphrases the context or an earlier node, adding no new information.
`Assumption`	A premise marked as not necessarily true, scoping the nodes that depend on it.
`Example`	A concrete instance illustrating a general concept introduced earlier.
`Reflection`	Subjective judgment or emotion about earlier nodes.
`Conclusion`	An intermediate or final answer to the input, asserted by the model.
Edge labels	
`infer`	Premise → result (logical deduction, calculation, or inductive hypothesis)
`execute`	`Planning` → the step that implements the plan.
`restate`	Original node → the node citing/restating the original node.
`elaborate-fact`	General `Fact` → a more specific `Fact` detailing it.
`exemplify`	Concept → an `Example` illustrating it.
`proceed`	Content that motivates a plan → `Planning`.
`verify`	Original node → `Planning` that initiates its verification.
`decompose`	Coarse `Planning` node → `Planning` with finer subplan or sub-case.
`backtrack`	`Planning` → alternative `Planning` (backtracking).
`positive`	Node → `Reflection` affirming it.
`negative`	Node → `Reflection` denying it or showing doubt.
`uncertain`	Node → `Reflection` expressing uncertainty or confusion.
`support`	Node → independently derived node asserting the same proposition.
`attack`	Node → node that contradicts or explicitly corrects it.

Table 2: Summary of 8 node labels and 14 edge labels of ReasoningFlow. Detailed definitions and examples can be found in Tables 4 and 5.

Appendix B includes a detailed comparison between ReasoningFlow and related works across LLM reasoning, computational linguistics, formal logic, and cognitive science.

3 ReasoningFlow schema

We introduce ReasoningFlow, a framework for annotating fine-grained semantic structures of reasoning traces.

We adopt a *directed acyclic graph* (DAG) structure with edges always flowing from earlier steps to later steps, resembling the left-to-right information flow in autoregressive LLMs (Ling et al., 2023; Bogdan et al., 2025). Compared to the projective Rhetorical Structure Theory-based trees (Carlson et al., 2001) and single-root argumentation trees (Stab and Gurevych, 2017), DAG provides both structural flexibility (e.g., crossing edges, one step having multiple successors) and a straightforward automatic annotation algorithm (Section 4.2).

Nodes. Nodes are contiguous, non-overlapping snippets that contain elementary reasoning steps. We primarily treat each sentence as a single node, but we divide a sentence into multiple nodes when two clauses are connected with distinct functional roles. For instance, if a step reads *"Therefore, x should be 17, but I should double-check."*, it is more natural to assign different roles to the first (calculating the answer) and the second half (planning the verification).

We define 8 node types based on their functional roles. The three core types are `Reasoning`, `Planning`, and `Reflection`. `Reasoning` nodes contain main building blocks like deduction and calculation, `Planning` nodes introduce the content of upcoming nodes, and `Reflection` nodes evaluate the correctness or express certainty on previous nodes. In addition, we define five special cases of `Reasoning` nodes, namely `Fact`, `Restatement`, `Assumption`, `Example`, and `Conclusion`, that indicate special roles for downstream applications. Table 4 lists full definitions and examples for all node

labels.

Edges. The next step is to annotate semantic relations between nodes as directed edges. All edges are constrained to flow left-to-right, uniquely connecting earlier nodes to later nodes in the sequence.

We define 14 edge labels of four major categories: Reason, Plan, Reflect, and Validate. Reason-related edges describe how the current step is derived from previous premises, plans, and facts. Plan-related edges show the intent of the `Planning` node, e.g., to proceed to a next step, or to verify the previous findings. Reflect-related edges show what `Reflection` nodes evaluate, along with the sentiment. Finally, Validate-related edges compare the propositional equivalence between distant nodes, deciding whether the following nodes `support` or `attack` the previous statement. Detailed definitions and examples for edge labels can be found in Table 5.

4 Dataset construction

4.1 Manual annotation

To validate the ReasoningFlow schema's clarity, we performed manual annotation with inter-annotator agreement evaluation. The manually annotated portion consists of 31 math, physics, and chemistry questions selected from NuminaMath (Zeng et al., 2025a) and STILL-2 (Min et al., 2024), and traces were generated by QwQ-32B-Preview (Qwen Team, 2024) with temperature 0.

Four of the authors participated in manual annotation, where two annotators were assigned to each trace. All annotators were provided with the same segmentation manually done by one of the authors. Two annotators individually completed the annotation without interacting, which was used for inter-annotator agreement analysis below. Then, annotators discussed the discrepancies in the annotation, which were incorporated in the guidelines, subtype definitions, and few-shot examples used for LLM-based annotation.

We measure two types of inter-annotator agreements, namely Node Classification (NC) and Edge Detection/Classification (EDC). NC measures whether both annotators selected the same node label (number of categories $k = 8$); EDC measures whether two annotators agree on whether the two nodes are connected or not, and the edge label if connected ($k = 15$; `no-edge` and 14 edge labels). The results show that annotators agree significantly on NC and EDC with Krippendorff's $\alpha > 0.8$ (Table 3), which is considered highly reliable (Krippendorff, 2004). This level of agreement indicates that the ReasoningFlow categories are well-defined and consistently interpretable across annotators.

Metric	Krippendorff's α	N
NC	0.8851	1,657
EDC	0.9193	122,630

Table 3: Inter-annotator agreement (Krippendorff's α) measured between four human annotators. Each example was assigned to two annotators. High $\alpha > 0.8$ validates the annotation schema of ReasoningFlow.

For node classification, the most common disagreement is whether a node should be classified as `Reasoning` or others. For instance, "*Given that Z is the midpoint, we can use the midpoint formula to find the coordinates of X.*" was annotated as `Reasoning` and `Planning` between different annotators. Edge classification disagreements commonly stemmed from node classification disagreements (e.g., `infer` or `execute` edge labels apply when the previous node is `Reasoning` or `Planning`, respectively).

4.2 Automatic annotation

We perform large-scale annotation of ReasoningFlow using an LLM-powered automatic annotation pipeline.

4.2.1 Base trace generation

We choose three representative datasets: AIME 2024 (Mathematical Association of America, 2024), GPQA-Diamond (Rein et al., 2023), and ArgKP (Bar-Haim et al., 2020). AIME 2024 contains 30 competition-level math problems. GPQA-Diamond is a benchmark for scientific knowledge and reasoning, including 198 problems across physics, chemistry, and biology. Finally, ArgKP is an argumentation benchmark with 24 debatable statements (e.g., *We should prohibit flag burning.*), where the objective is to choose a stance (agree/disagree) and provide reasons.

A total of five models were used to collect the reasoning trace. For LRMs, we choose three representative models: `DeepSeek-R1` (Guo et al., 2025, 671B), `QwQ-32B` (Qwen Team, 2024), and `GPT-oss-120B` (OpenAI, 2025, Reasoning effort: Medium). We also include two non-reasoning models: `DeepSeek-V3` (DeepSeek-AI, 2024, 671B) and `Qwen2.5-32B-Instruct` (Yang et al., 2024). We use greedy decoding (temperature 0) for all models.

As a result, we obtain 1,260 reasoning traces across five models and three datasets.

4.2.2 Annotation pipeline

The annotation pipeline consists of three stages: node segmentation, node classification, and edge detection/classification.

During node segmentation, we instruct LLMs to segment raw reasoning traces into nodes, providing definitions and examples of where to segment and where not to. In the second stage (node classification), we prompt LLMs with node label definitions and representative examples.

Finally, in the edge detection/classification stage, we find all edges and their relation labels in a single pass. Since the unconstrained DAG structure of ReasoningFlow admits up to $O(N^2)$ edges, we ask LLMs to identify incoming edges for a single node for each inference to reduce output length and improve annotation performance. Details are presented in Appendix C.1.

We used Gemini-3.1-Flash for node annotation and Gemini-3-Pro for edge annotation, as they achieve the best F1-scores on the manually annotated set with NC (0.865) and EDC (0.646), respectively; see Appendix C.2 for scores from different models. Furthermore, the authors manually verified 30 of the automatic annotations by editing LLM predictions following Mukherjee et al. (2025); Zeng et al. (2025b), achieving F1 score of NC 0.909 and EDC 0.869 (Appendix C.4).

4.3 Node quality annotation

We annotate the *quality* of ReasoningFlow nodes for further analyses (Sections 6-7).

For reasoning tasks (AIME, GPQA), we define the quality of a node as *validity*, or logical correctness (Lightman et al., 2024; Lee and Hockenmaier, 2025). For each `Reasoning` node, we employ LLM-as-a-judge (Gemini-3-Flash) to annotate whether the step is logically correct. We provide connected preceding nodes to the judge instead of the full context to reduce input length, following existing works (Ling et al., 2023; Mukherjee et al., 2025).

For the argumentation task (ArgKP), we leverage the AQR dataset (Gretz et al., 2020), which includes 30k crowdsourced arguments corresponding to ArgKP dataset's topics and human-annotated quality scores (0-1) for each argument. Specifically, we identify arguments in AQR that are equivalent to each `Fact` and `Reasoning` nodes using Gemini-3-Flash, and use the average score of matching arguments as the node quality.

Appendix D includes detailed methods and manual verification results.

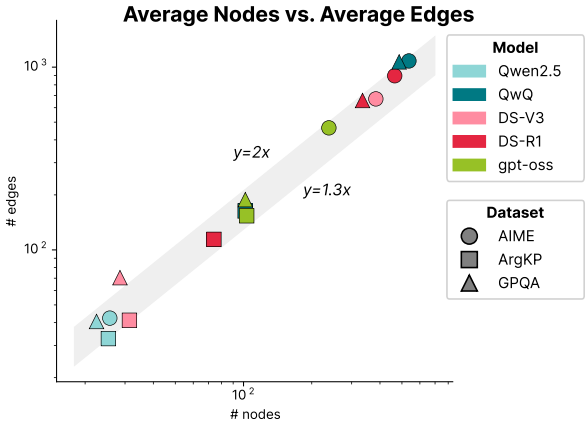


Figure 2: Average number of nodes and edges per (model, dataset). While the graph size varies, the average degree (edge/node) remains in 1.3-2.0 (gray region).

5 ResaoningFlow statistics

5.1 Nodes/edges count

The size of the graph varies considerably across models and datasets; QwQ generates an average of 455.6 nodes for AIME, while Qwen2.5-32B generates only 29.2 nodes for ArgKP. In contrast, the average degree (number of incoming edges per node) remains relatively stable across configurations, consistently falling between 1.3 and 2.0 (Figure 2). This is consistent with prior findings in non-reasoning models, where each step typically draws on only a small number of premises (Ling et al., 2023; Mukherjee et al., 2025).

5.2 Comparison between models/domains

Finding 1

Different LRMs exhibit similar ReasoningFlow structures (triplet distributions).

To investigate how reasoning structures vary across models and domains, we compare the distribution of (node label, edge label, node label) **triplets** extracted from ReasoningFlow graphs. Each triplet type relates to a specific reasoning behavior; e.g., `Reasoning` – `verify` → `Planning` triplets indicate verification, while `Assumption` – `attack` → `Reasoning` corresponds to proof-by-contradiction. For each (model, domain) pair, we compute the distribution of these triplets.

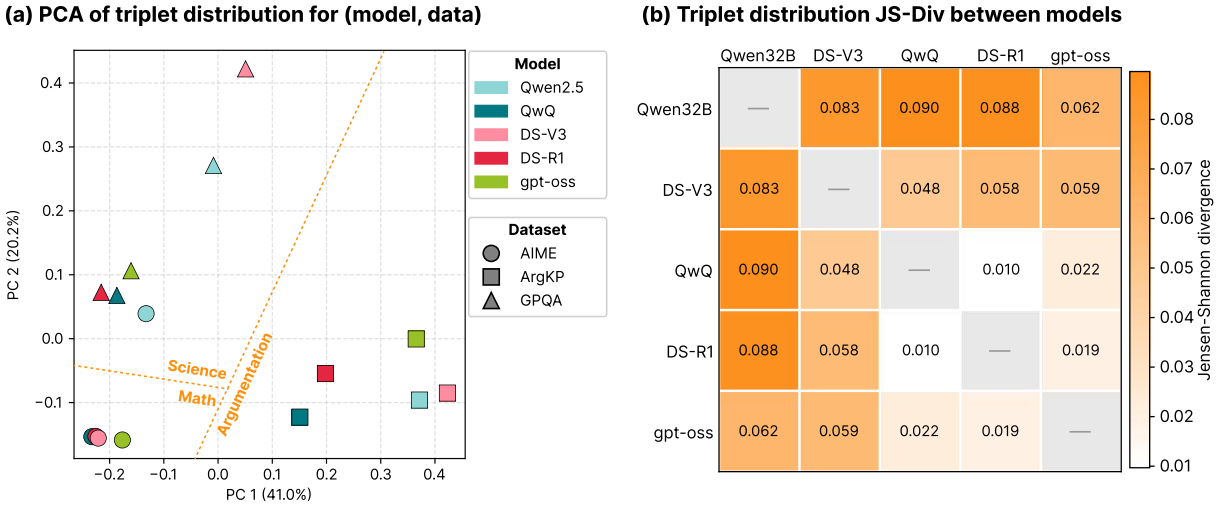


Figure 3: (a) Hellinger Principal Component Analysis (PCA) plot (Lebret and Collobert, 2014) of triplet probability distribution, showing clusters of datasets over clusters of models. (b) Jensen-Shannon Divergence of triplet distributions between models.

We apply Hellinger Principal Component Analysis (PCA) for explainable clustering of the triplet distribution of (model, dataset) pairs (Ding and He, 2004; Lebret and Collobert, 2014) (Figure 3(a)). The main clusters form based on the domain rather than the generated model, indicating that different reasoning tasks elicit unique reasoning structures.

Principal components further reveal important features distinguishing the datasets. Argumentation traces include several subarguments listed in a sequence, which is realized by frequent `Planning` – `proceed` → `Planning`. Math reasoning (AIME) involves more deductive reasoning steps `Reasoning` – `infer` → `Reasoning`, while science reasoning (GPQA) triggers more solution-level verification `Conclusion` – `support` → `Conclusion`. Refer to Appendix E for details on PCA analysis.

Across models, reasoning model (QwQ, DS-R1, GPT-oss) traces are more structurally similar to one another than their base model counterparts, particularly on reasoning-heavy tasks. Figure 3(b) displays the Jensen-Shannon divergence (JS-Div; Fuglede and Topsøe, 2004) between the triplet distribution of different models, all datasets averaged. While Qwen2.5-32B and DeepSeek-V3 exhibit substantial structural differences (JS-Div= 0.083), their corresponding reasoning model checkpoints (QwQ and DeepSeek-R1) are far more similar (JS-Div= 0.010). This observation suggests that different reasoning models, despite being trained with different base models and data, exhibit structural similarity in their reasoning traces.

6 ReasoningFlow and reasoning behaviors

6.1 Local verification

Finding 2.1

Local verification effectively corrects erroneous steps before the final answer is derived.

LRMs engage in self-verification by assessing the correctness of prior reasoning steps and revising them as needed. Previous work focused on *global* verification, where the LRM starts verification on the entire solution after finding the first answer (Gandhi et al., 2025; Marjanovic et al., 2026). Most of these global verifications conclude by simply restating the original answer, regardless of its correctness (Zhao et al., 2025; Liao et al., 2025).

However, these studies overlook **local verification**, in which the model detects a potential error mid-reasoning and corrects it within the next few steps (Figure 4). Local verification is universal across different LRMs and datasets, and happens more frequently than global verification of the final answer (19.1 times/trace vs. 6.1 times/trace).

We further analyze local verification cases where the model explicitly *corrects* previous nodes (`attack` edge). These corrections are more likely to be correct reasoning steps (Figure 5(a)), and are significantly more likely to be used to derive the correct final answer(`Conclusion` node) than their original statements (Figure 5(b)). This observation shows that local verifications directly steer the

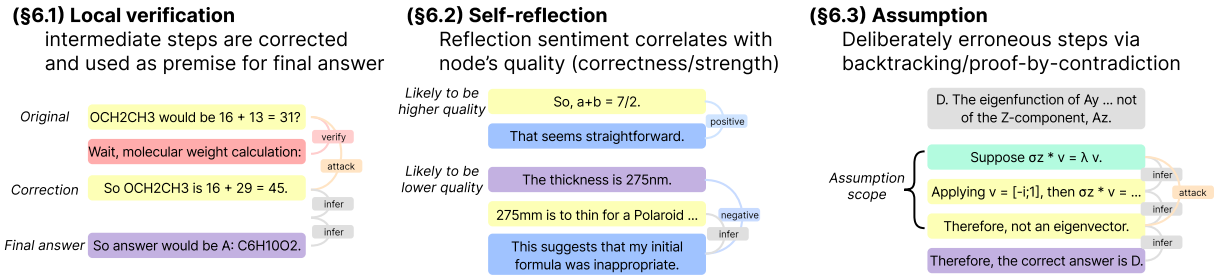


Figure 4: Examples of three fine-grained reasoning behaviors (local verification, self-reflection, and assumption).

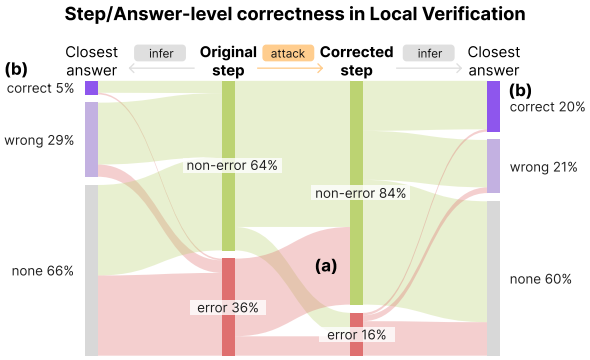


Figure 5: Sankey plot showing how local verification with explicit correction (`attack`) affects subsequent reasoning. (a) Local verification, when used, corrects step-level errors (64% → 84%). (b) Corrected steps are more likely to be used to derive the correct final answer than original steps (20% vs. 5%).

following reasoning process towards the correct answer. Improving local verification in LRMs could improve both efficiency, by correcting errors before a first final answer is reached, and monitorability, by localizing where those corrections occur.

6.2 Self-reflection

Finding 2.2

Self-reflection sentiment correlates with the node quality, enabling a new direction for reasoning monitorability.

Self-reflection consists of meta-evaluation of previous nodes (*"Looks good"*) or emotions/impressions (*"but I am not sure."*). Three edge labels in ReasoningFlow (`positive` , `uncertain` , `negative`) capture this behavior along with the reflection sentiment. With ReasoningFlow, we analyze whether LRMs' self-reflection faithfully reflects the quality of the reflected statement.

We observe a clear trend between node quality and self-reflection sentiment in reasoning models.

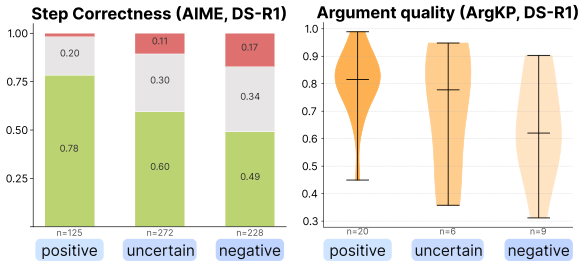


Figure 6: Self-reflection sentiment correlates with node quality (correctness, argument quality). For step correctness, green/gray/red corresponds to correct nodes, propagated errors, and direct errors, respectively (Appendix D).

Across three LRMs in AIME/GPQA, nodes with `positive` reflection are 78.1% correct, while step correctness decreases to 66.2% in `uncertain` and to 45.6% in `negative` . Similar trends are observed in argumentation quality; positive reflections are associated with stronger arguments, while uncertain/negative reflections indicate weaker arguments. Overall, this indicates that self-reflection phrases are not just fillers (Wang et al., 2025a) but reflect the node quality, opening up a new possibility for monitoring internal beliefs of LRMs.

6.3 Assumption

Finding 2.3

ReasoningFlow identifies proof-by-contradiction and DFS, enabling evaluation of hypothetical reasoning abilities.

Assuming facts that might not be true is a hallmark of human intelligence (Van Hoeck et al., 2015). In reasoning, the most popular usage of assumptions is *proof-by-contradiction*: deriving a contradiction from a deliberately false assumption. Another common use case is *depth-first search (DFS)*: when encountering mutually exclusive branches, assuming one option at a time instead of considering all options simultaneously.

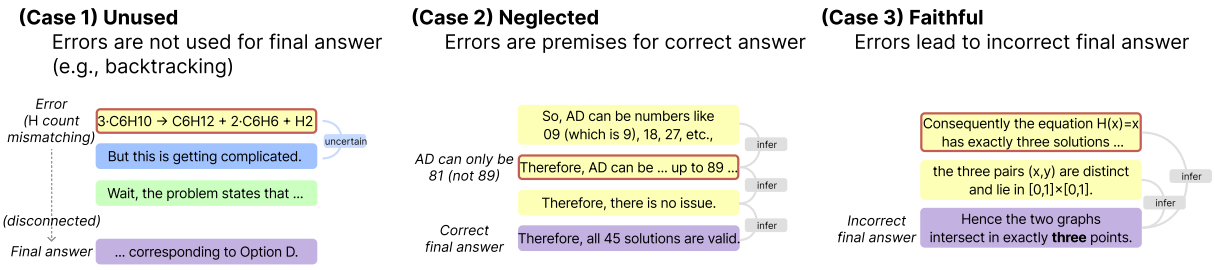


Figure 7: Examples of three types of error handling in LRMs (unused, neglected, and faithful error propagation).

In ReasoningFlow, proof-by-contradiction is realized by `Assumption` – `attack` → `Reasoning` triplets that show assumption being refuted by following nodes (Figure 4), while DFS is most frequently characterized by `Assumption` – `proceed` → `Assumption` triplets that chain multiple assumptions together to enable exhaustive search. ReasoningFlow identifies 161 proof-by-contradiction and 2,514 DFS-related patterns, covering 41.9% of the total `Assumption` nodes.

Deliberate errors that follow a false assumption should be treated differently from normal errors. Evaluating the hypothetical reasoning ability of LLMs, not by the truth value of the statement but by the plausibility of the assumption or the exhaustiveness of DFS, remains underexplored for evaluating higher-order reasoning of LRM.

7 ReasoningFlow and stepwise evaluation

Finding 3

Most erroneous steps in LRMs do not lead to incorrect final answers.

Stepwise evaluation identifies erroneous steps within a reasoning trace, often using classifiers (Process Reward Models) or LLM-as-a-judge (Lee and Hockenmaier, 2025). Early works implicitly assumed that a logical error leads to an incorrect final answer, e.g., Best-of-N sampling that filters out any traces containing errors (Ling et al., 2023; Lightman et al., 2024). However, empirical evidence suggests that LLMs can produce erroneous intermediate steps and yet still arrive at correct conclusions (Yee et al., 2024; Kim et al., 2025), likely because not all steps are used to derive the final answer.

Since ReasoningFlow tracks the premises of every step, we can determine whether a given step contributed to the final answer. Figure 7 illustrates three possible relations between an error and the final answer: the error plays no role in any subsequent conclusion (*Unused*), the error is used as a premise of a correct answer (*Neglected*), or the error propagates to produce an incorrect answer (*Faithful*).

ReasoningFlow reveals that in LRMs, only 14.4% of the erroneous nodes propagate to incorrect final answers. First, 79.6% of the errors do not connect to the final answer, indicating that most errors are *unused* during reasoning. This occurs particularly during backtracking, when the LLM did not develop a final answer while exploring that direction. Second, the remaining 6.0% of the errors (32.8% of the errors connected to `Conclusion` nodes) lead to a *correct* final answer, indicating that the error was *neglected* within the core arguments. Detailed statistics for each (model, dataset) pair are presented in Appendix G.

These unused and neglected errors together prove that the logical link between reasoning errors and incorrect final answers is often weak, highlighting the unfaithfulness of LRMs in handling errors. We argue that stepwise evaluation can be improved by incorporating the discourse structure of traces to track how errors propagate downstream.

8 ReasoningFlow and mechanistic interpretability

Finding 4

Mechanistically measured causal step-to-step dependencies do not align with language-level discourse relations.

Mechanistic interpretation of trace structures. Thought Anchors (Bogdan et al., 2025) proposes measuring the causal dependency between reasoning steps by masking attentions. To measure the dependency between steps s_i and s_j ($i < j$), it first obtains s_j's token probability distribution. Then, it recomputes the token probabilities after masking all tokens in s_i. Finally, the causal dependency score is defined as the average log KL-divergence

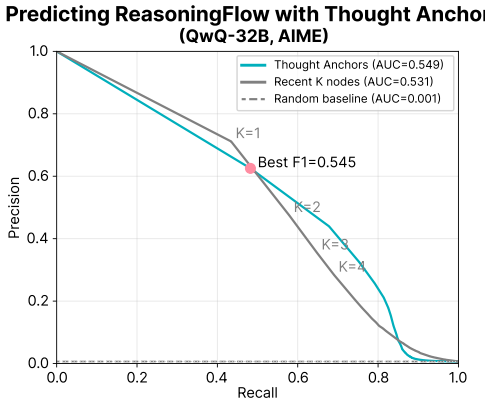


Figure 8: Precision-Recall curve of using Thought Anchors' scores to predict ReasoningFlow edges. Full results are presented in Figure 12.

between the probability distribution over all tokens in step s_j. Qualitatively, a higher score implies that s_i casts a significant effect on the generation of s_j, while a lower score implies the opposite.

Results. Figure 8 shows the precision-recall graph of using Thought Anchors' scores to predict ReasoningFlow edges in (QwQ, AIME). While Thought Anchors outperform the random baseline, they are not significantly better compared to greedily choosing the closest $K = 1, 2, 3, ...$ nodes. This indicates that the distance between two nodes is the primary driver of the positive correlation.

Comparing Thought Anchors and ReasoningFlow reveals the discrepancy between the text-level structures and their mechanistic interpretation. This aligns with prior work on synthetic logical benchmarks (Zhong et al., 2026) and in-context retrieval (Du et al., 2025), which found that greater distance between the steps alone introduces significant noise into semantic dependency representations. Aligning internal representations with surface semantics is a crucial challenge for faithful LRMs, where ReasoningFlow can serve as a tool for understanding the language part of the causal reasoning procedure.

9 Conclusion

We propose ReasoningFlow, a comprehensive framework for annotating discourse structures in reasoning traces. ReasoningFlow dataset comprises 1.3k manually and automatically annotated traces with fine-grained node and edge labels. ReasoningFlow can be used to show structural similarity between LRMs, discover novel reasoning behaviors to the sub-sentence level, assess how erroneous steps causally affect the final answer, and identify the gap between mechanistic and discourse structures in reasoning models.

Beyond the analyses presented here, we believe ReasoningFlow can serve as a general, human-interpretable lens for studying the reasoning capabilities of LRMs. Promising directions include examining how reasoning behaviors emerge over the course of training (Ettinger et al., 2025; Wang et al., 2025b), how LRMs adapt their reasoning under constrained inference-time compute budgets (Wen et al., 2025), and how they resolve conflicting knowledge (Lin et al., 2025).

Limitations

ReasoningFlow does not include manual verification of *all* data (1.2k traces) due to significant annotation cost. To ensure the quality of the data and downstream analyses, we present partial manual verification results throughout the data annotation pipeline (Section 4) in Appendix C-D.

ReasoningFlow includes three representative open-weight LRMs (QwQ, DeepSeek-R1, GPT-oss). Closed-source models (e.g., o1, Gemini, Claude) or distilled models (e.g., DeepSeek-R1-Distill (Guo et al., 2025), OpenThoughts (Guha et al., 2025)) might demonstrate different behaviors when analyzed with ReasoningFlow. However, the fact that independently trained reasoning models exhibit structural similarity (Section 5) suggests that other LRMs will likely exhibit a similar distribution.

The current automatic annotation pipeline for ReasoningFlow is costly. At this moment, we prioritize the dataset quality over the inference cost by using API-based frontier models. The automatically annotated data can be used to train smaller, specialized models, which will significantly reduce the annotation cost and increase the availability of ReasoningFlow.

Acknowledgements

We gratefully acknowledge LBOX for their generous research gift. We also thank Sagnik Mukherjee, Yukyung Lee, and Takyoung Kim for their valuable discussions and insights.

References

Nishant Balepur, Rachel Rudinger, and Jordan Lee Boyd-Graber. 2025. Which of These Best Describes

Multiple Choice Evaluation with LLMs? a) Forced B) Flawed C) Fixable D) All of the Above. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, pages 3394–3418. Association for Computational Linguistics.

Guangsheng Bao, Hongbo Zhang, Cunxiang Wang, Linyi Yang, and Yue Zhang. 2025. How Likely Do LLMs with CoT Mimic Human Reasoning? In *Proceedings of the 31st International Conference on Computational Linguistics, COLING 2025, Abu Dhabi, UAE, January 19-24, 2025*, pages 7831–7850. Association for Computational Linguistics.

Roy Bar-Haim, Lilach Eden, Roni Friedman, Yoav Kantor, Dan Lahav, and Noam Slonim. 2020. From Arguments to Key Points: Towards Automatic Argument Summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, ACL 2020, Online, July 5-10, 2020*, pages 4029–4039. Association for Computational Linguistics.

Paul C. Bogdan, Uzay Macar, Neel Nanda, and Arthur Conmy. 2025. Thought Anchors: Which LLM Reasoning Steps Matter? *CoRR*, abs/2506.19143.

Lynn Carlson, Daniel Marcu, and Mary Ellen Okurovsky. 2001. Building a Discourse-Tagged Corpus in the Framework of Rhetorical structure Theory. In *Proceedings of the SIGDIAL 2001 Workshop, The 2nd Annual Meeting of the Special Interest Group on Discourse and Dialogue, Saturday, September 1, 2001 to Sunday, September 2, 2001, Aalborg, Denmark*. The Association for Computer Linguistics.

Edward Y. Chang, Yuxuan Tong, Morry Niu, Graham Neubig, and Xiang Yue. 2025. Demystifying Long Chain-of-Thought Reasoning in LLMs. *CoRR*, abs/2502.03373.

Yanda Chen, Joe Benton, Ansh Radhakrishnan, Jonathan Uesato, Carson Denison, John Schulman, Arushi Somani, Peter Hase, Misha Wagner, Fabien Roger, Vladimir Mikulik, Samuel R. Bowman, Jan Leike, Jared Kaplan, and Ethan Perez. 2025. Reasoning Models Don't Always Say What They Think. *CoRR*, abs/2505.05410.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *CoRR*, abs/2110.14168.

Antonia Creswell, Murray Shanahan, and Irina Higgins. 2023. Selection-Inference: Exploiting Large Language Models for Interpretable logical Reasoning. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.

Bhavana Dalvi, Peter Jansen, Oyvind Tafjord, Zhengnan Xie, Hannah Smith, Leighanna Pipatanangkura, and Peter Clark. 2021. Explaining Answers with Entailment Trees. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021, Virtual Event / Punta Cana, Dominican Republic, 7-11 November, 2021*, pages 7358–7370. Association for Computational Linguistics.

DeepSeek-AI. 2024. DeepSeek-V3 Technical Report. *CoRR*, abs/2412.19437.

Chris H. Q. Ding and Xiaofeng He. 2004. \emphK-means clustering via principal component analysis. In *Machine Learning, Proceedings of the Twenty-first International Conference (ICML 2004), Banff, Alberta, Canada, July 4-8, 2004*. ACM.

Yufeng Du, Minyang Tian, Srikanth Ronanki, Subendhu Rongali, Sravan Babu Bodapati, Aram Galstyan, Azton Wells, Roy Schwartz, Eliu A. Huerta, and Hao Peng. 2025. Context Length Alone Hurts LLM Performance Despite Perfect Retrieval. In *Findings of the Association for Computational Linguistics: EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 23281–23298. Association for Computational Linguistics.

Allyson Ettinger, Amanda Bertsch, Bailey Kuehl, David Graham, David Heineman, Dirk Groeneveld, Faeze Brahman, Finbarr Timbers, Hamish Ivison, Jacob Morrison, Jake Poznanski, Kyle Lo, Luca Soldaini, Matt Jordan, Mayee F. Chen, Michael Noukhovitch, Nathan Lambert, Pete Walsh, Pradeep Dasigi, and 48 others. 2025. Olmo 3. *CoRR*, abs/2512.13961.

Bent Fuglede and Flemming Topsøe. 2004. Jensen-Shannon divergence and Hilbert space embedding. In *Proceedings of the 2004 IEEE International Symposium on Information Theory, ISIT 2004, Chicago Downtown Marriott, Chicago, Illinois, USA, June 27 - July 2, 2004*, page 31. IEEE.

Kanishk Gandhi, Ayush Chakravarthy, Anikait Singh, Nathan Lile, and Noah D. Goodman. 2025. Cognitive Behaviors that Enable Self-Improving Reasoners, or, Four habits of Highly Effective STaRs. *CoRR*, abs/2503.01307.

Shai Gretz, Roni Friedman, Edo Cohen-Karlik, Assaf Toledo, Dan Lahav, Ranit Aharonov, and Noam Slonim. 2020. A Large-Scale Dataset for Argument Quality Ranking: Construction and analysis. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*, pages 7805–7813. AAAI Press.

Jiawei Gu, Xuhui Jiang, Zhichao Shi, Hexiang Tan, Xuehao Zhai, Chengjin Xu, Wei Li, Yinghan Shen, Shengjie Ma, Honghao Liu, Yuanzhuo Wang, and Jian Guo. 2024. A Survey on LLM-as-a-Judge. *CoRR*, abs/2411.15594.

Etash Kumar Guha, Ryan Marten, Sedrick Keh, Negin Raoof, Georgios Smyrnis, Hritik Bansal, Marianna Nezhurina, Jean Mercat, Trung Vu, Zayne Sprague, Ashima Suvarna, Benjamin Feuer, Liangyu Chen, Zaid Khan, Eric Frankel, Sachin Grover, Caroline Choi, Niklas Muennighoff, Shiye Su, and 31 others. 2025. OpenThoughts: Data Recipes for Reasoning Models. *CoRR*, abs/2506.04178.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, and 175 others. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nat.*, 645(8081):633–638.

Ivan Habernal, Daniel Faber, Nicola Recchia, Sebastian Bretthauer, Iryna Gurevych, Indra Spiecker genannt Döhmann, and Christoph Burchard. 2024. Mining legal arguments in court decisions. *Artif. Intell. Law*, 32(3):1–38.

Simeng Han, Aaron Yu, Rui Shen, Zhenting Qi, Martin Riddell, Wenfei Zhou, Yujie Qiao, Yilun Zhao, Semih Yavuz, Ye Liu, Shafiq Joty, Yingbo Zhou, Caiming Xiong, Dragomir Radev, Rex Ying, and Arman Cohan. 2024. P-FOLIO: Evaluating and Improving Logical Reasoning with Abundant human-Written Reasoning Chains. In *Findings of the Association for Computational Linguistics: EMNLP 2024, Miami, Florida, USA, November 12-16, 2024*, pages 16553–16565. Association for Computational Linguistics.

Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. 2024. Training Large Language Models to Reason in a Continuous Latent Space. *CoRR*, abs/2412.06769.

Yancheng He, Shilong Li, Jiaheng Liu, Weixun Wang, Xingyuan Bu, Ge Zhang, Z. Y. Peng, Zhaoxiang Zhang, Zhicheng Zheng, Wenbo Su, and Bo Zheng. 2025. Can Large Language Models Detect Errors in Long Chain-of-Thought Reasoning? In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, pages 18468–18489. Association for Computational Linguistics.

Gangwei Jiang, Yahui Liu, Zhaoyi Li, Wei Bi, Fuzheng Zhang, Linqi Song, Ying Wei, and Defu Lian. 2025. What Makes a Good Reasoning Chain? Uncovering Structural Patterns in Long Chain-of-Thought Reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 6490–6514. Association for Computational Linguistics.

Priyanka Kargupta, Shuyue Stella Li, Haocheng Wang, Jinu Lee, Shan Chen, Orevaoghene Ahia, Dean Light, Thomas L. Griffiths, Max Kleiman-Weiner, Jiawei Han, Asli Celikyilmaz, and Yulia Tsvetkov. 2025. Cognitive Foundations for Reasoning and Their Manifestation in LLMs. *CoRR*, abs/2511.16660.

Mehran Kazemi, Najoung Kim, Deepti Bhatia, Xin Xu, and Deepak Ramachandran. 2023. LAMBADA: Backward Chaining for Automated Reasoning in Natural Language. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9-14, 2023*, pages 6547–6568. Association for Computational Linguistics.

Gerard Kempen and Edward Hoenkamp. 1987. An Incremental Procedural Grammar for Sentence Formulation. *Cogn. Sci.*, 11(2):201–258.

Seungone Kim, Ian Wu, Jinu Lee, Xiang Yue, Seongyun Lee, Mingyeong Moon, Kiril Gashteovski, Carolin Lawrence, Julia Hockenmaier, Graham Neubig, and Sean Welleck. 2025. Scaling Evaluation-time Compute with Reasoning Models as Process Evaluators. *CoRR*, abs/2503.19877.

Klaus Krippendorff. 2004. Reliability in Content Analysis.: Some Common Misconceptions and Recommendations. *Human Communication Research*, 30(3):411–433.

Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, Kamile Lukosiute, Karina Nguyen, Newton Cheng, Nicholas Joseph, Nicholas Schiefer, Oliver Rausch, Robin Larson, Sam McCandlish, Sandipan Kundu, and 11 others. 2023. Measuring Faithfulness in Chain-of-Thought Reasoning. *CoRR*, abs/2307.13702.

Anne Lauscher, Goran Glavas, and Simone Paolo Ponzetto. 2018. An Argument-Annotated Corpus of Scientific Publications. In *Proceedings of the 5th Workshop on Argument Mining, ArgMining@EMNLP 2018, Brussels, Belgium, November 1, 2018*, pages 40–46. Association for Computational Linguistics.

Rémi Lebret and Ronan Collobert. 2014. Word Embeddings through Hellinger PCA. In *Proceedings of the 14th Conference of the European Chapter of the Association for Computational Linguistics, EACL 2014, April 26-30, 2014, Gothenburg, Sweden*, pages 482–490. The Association for Computer Linguistics.

Jinu Lee and Julia Hockenmaier. 2025. Evaluating Step-by-step Reasoning Traces: A Survey. In *Findings of the Association for Computational Linguistics: EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 1789–1814. Association for Computational Linguistics.

Jinu Lee and Wonseok Hwang. 2025. SymBa: Symbolic Backward Chaining for Structured Natural Language Reasoning. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2025 - Volume 1:*

Long Papers, Albuquerque, New Mexico, USA, April 29 - May 4, 2025, pages 2468–2484. Association for Computational Linguistics.

Jinu Lee, Sagnik Mukherjee, Dilek Hakkani-Tur, and Julia Hockenmaier. 2025. ReasoningFlow: Semantic Structure of Complex Reasoning Traces. *CoRR*, abs/2506.02532.

Yifei Li, Zeqi Lin, Shizhuo Zhang, Qiang Fu, Bei Chen, Jian-Guang Lou, and Weizhu Chen. 2023. Making Language Models Better Reasoners with Step-Aware Verifier. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2023, Toronto, Canada, July 9-14, 2023*, pages 5315–5333. Association for Computational Linguistics.

Baohao Liao, Xinyi Chen, Sara Rajaee, Yuhui Xu, Christian Herold, Anders S{\o}gaard, Maarten de Rijke, and Christof Monz. 2025. Lost at the Beginning of Reasoning. *CoRR*, abs/2506.22058.

Vladimir Lifschitz. 2019. Answer Set Programming. ISBN: 9783030246570 9783030246587.

Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2024. Let's Verify Step by Step. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.

Chenyu Lin, Yilin Wen, Du Su, Hexiang Tan, Fei Sun, Muhan Chen, Chenfu Bao, and Zhonghou Lyu. 2025. Resisting Contextual Interference in RAG via Parametric-Knowledge Reinforcement. *arXiv preprint*. ArXiv:2506.05154 [cs] version: 2.

Zhan Ling, Yunhao Fang, Xuanlin Li, Zhiao Huang, Mingu Lee, Roland Memisevic, and Hao Su. 2023. Deductive Verification of Chain-of-Thought Reasoning. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

Ryan Liu, Jiayi Geng, Addison J. Wu, Ilia Sucholutsky, Tania Lombrozo, and Thomas L. Griffiths. 2025. Mind Your Step (by Step): Chain-of-Thought can Reduce Performance on Tasks where Thinking Makes Humans Worse. In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*. PMLR / OpenReview.net.

William C. Mann and Sandra A. Thompson. 1988. Rhetorical Structure Theory: Toward a functional theory of text organization. *Text - Interdisciplinary Journal for the Study of Discourse*, 8(3).

Daniel Marcu. 2000. The Rhetorical Parsing of Unrestricted Texts: A Surface-Based Approach. *Comput. Linguistics*, 26(3):395–448.

Sara Vera Marjanovic, Arkil Patel, Vaibhav Adlakha, Milad Aghajohari, Parishad BehnamGhader, Mehar Bhatia, Aditi Khandelwal, Austin Kraft, Benno Krojer, Xing Han Lu, Nicholas Meade, Dongchan Shin, Amirhossein Kazemnejad, Gaurav Kamath, Marius Mosbach, Karolina Stanczak, and Siva Reddy. 2026. DeepSeek-R1 Thoughtology: Let's think about LLM reasoning. *Trans. Mach. Learn. Res.*, 2026.

Mathematical Association of America. 2024. American Invitational Mathematics Examination I/II.

Yingqian Min, Zhipeng Chen, Jinhao Jiang, Jie Chen, Jia Deng, Yiwen Hu, Yiru Tang, Jiapeng Wang, Xiaoxue Cheng, Huatong Song, Wayne Xin Zhao, Zheng Liu, Zhongyuan Wang, and Ji-Rong Wen. 2024. Imitate, Explore, and Self-Improve: A Reproduction Report on Slow-thinking reasoning Systems. *CoRR*, abs/2412.09413.

Mathieu Morey, Philippe Muller, and Nicholas Asher. 2017. How much progress have we made on RST discourse parsing? A replication study of recent results on the RST-DT. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing, EMNLP 2017, Copenhagen, Denmark, September 9-11, 2017*, pages 1319–1324. Association for Computational Linguistics.

Sagnik Mukherjee, Abhinav Chinta, Takyoung Kim, Tarun Anoop Sharma, and Dilek Hakkani-Tur. 2025. Premise-Augmented Reasoning Chains Improve Error Identification in math reasoning with LLMs. In *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*. PMLR / OpenReview.net.

Thanh-Tung Nguyen, Xuan-Phi Nguyen, Shafiq Joty, and Xiaoli Li. 2021. RST Parsing from Scratch. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1613–1625, Online. Association for Computational Linguistics.

OpenAI. 2025. gpt-oss-120b \& gpt-oss-20b Model Card. *CoRR*, abs/2508.10925.

Rashmi Prasad, Nikhil Dinesh, Alan Lee, Eleni Miltsakaki, Livio Robaldo, Aravind K. Joshi, and Bonnie L. Webber. 2008. The Penn Discourse TreeBank 2.0. In *Proceedings of the International Conference on Language Resources and Evaluation, LREC 2008, 26 May - 1 June 2008, Marrakech, Morocco*. European Language Resources Association.

Qwen Team. 2024. QwQ: Reflect Deeply on the Boundaries of the Unknown.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2023. GPQA: A Graduate-Level Google-Proof Q\&A Benchmark. *CoRR*, abs/2311.12022.

Christian Stab and Iryna Gurevych. 2017. Parsing Argumentation Structures in Persuasive Essays. *Comput. Linguistics*, 43(3):619–659.

Oyvind Tafjord, Bhavana Dalvi, and Peter Clark. 2021. ProofWriter: Generating Implications, Proofs, and Abductive Statements over Natural Language. In *Findings of the Association for Computational Linguistics: ACL/IJCNLP 2021, Online Event, August 1-6, 2021*, pages 3621–3634. Association for Computational Linguistics.

Jonathan Uesato, Nate Kushman, Ramana Kumar, H. Francis Song, Noah Y. Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving math word problems with process- and outcome-based feedback. *CoRR*, abs/2211.14275.

Nicole Van Hoeck, Patrick D. Watson, and Aron K. Barbey. 2015. Cognitive neuroscience of human counterfactual reasoning. *Frontiers in Human Neuroscience*, 9:420.

Douglas Walton, Christopher Reed, and Fabrizio Macagno. 2008. Argumentation Schemes. Cambridge University Press. Edition: 1.

Chenlong Wang, Yuanning Feng, Dongping Chen, Zhaoyang Chu, Ranjay Krishna, and Tianyi Zhou. 2025a. Wait, We Don't Need to "Wait"! Removing Thinking Tokens Improves Reasoning efficiency. In *Findings of the Association for Computational Linguistics: EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 7459–7482. Association for Computational Linguistics.

Haozhe Wang, Qixin Xu, Che Liu, Junhong Wu, Fangzhen Lin, and Wenhu Chen. 2025b. Emergent Hierarchical Reasoning in LLMs through Reinforcement Learning. *CoRR*, abs/2509.03646.

Hao Wen, Xinrui Wu, Yi Sun, Feifei Zhang, Liye Chen, Jie Wang, Yunxin Liu, Yunhao Liu, Ya-Qin Zhang, and Yuanchun Li. 2025. BudgetThinker: Empowering Budget-aware LLM Reasoning with Control tokens. *CoRR*, abs/2508.17196.

Jan Wielemaker. 2003. An Overview of the SWI-Prolog Programming Environment. In *Proceedings of the 13th International Workshop on Logic Programming Environments, Tata Institute of Fundamental Research, Mumbai, India, December 8, 2003*, pages 1–16. Katholieke Universiteit Leuven, Department of Computer Science, Celestijnenlaan 200A, B-3001 Heverlee (Belgium).

Bin Xie, Bingbing Xu, Xueyun Tian, Yilin Chen, and Huawei Shen. 2026. Towards Robust Process Reward Modeling via Noise-aware Learning. *CoRR*, abs/2601.12748.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, and 22 others. 2024. Qwen2.5 Technical Report. *CoRR*, abs/2412.15115.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.

Evelyn Yee, Alice Li, Chenyu Tang, Yeon Ho Jung, Ramamohan Paturi, and Leon Bergen. 2024. Dissociation of Faithful and Unfaithful Reasoning in LLMs. *CoRR*, abs/2405.15092.

Changyu Zeng, Yifan Wang, Zimu Wang, Wei Wang, Zhengni Yang, Muyi Bao, Jimin Xiao, Anh Nguyen, and Yutao Yue. 2025a. NUMINA: A Natural Understanding Benchmark for Multi-dimensional intelligence and Numerical Reasoning Abilities. In *Findings of the Association for Computational Linguistics: EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 22575–22590. Association for Computational Linguistics.

Yuchen Zeng, Shuibai Zhang, Wonjun Kang, Shutong Wu, Lynnix Zou, Ying Fan, Heeju Kim, Ziqian Lin, Jungtaek Kim, Hyung Il Koo, Dimitris Papailiopoulos, and Kangwook Lee. 2025b. ReJump: A Tree-Jump Representation for Analyzing and Improving LLM reasoning. *CoRR*, abs/2512.00831.

Duzhen Zhang, Zhong-Zhi Li, Ming-Liang Zhang, Jiaxin Zhang, Zengyan Liu, Yuxuan Yao, Haotian Xu, Junhao Zheng, Xiuyi Chen, Yingying Zhang, Fei Yin, Jiahua Dong, Zhijiang Guo, Le Song, and Cheng-Lin Liu. 2026. From System 1 to System 2: A Survey of Reasoning Large Language models. *IEEE Trans. Pattern Anal. Mach. Intell.*, 48(3):3335–3354.

Zhenru Zhang, Chujie Zheng, Yangzhen Wu, Beichen Zhang, Runji Lin, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2025. The Lessons of Developing Process Reward Models in Mathematical Reasoning. In *Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, pages 10495–10516. Association for Computational Linguistics.

Jiachen Zhao, Yiyou Sun, Weiyan Shi, and Dawn Song. 2025. Can Aha Moments Be Fake? Identifying True and Decorative Thinking steps in Chain-of-Thought. *CoRR*, abs/2510.24941.

Chujie Zheng, Zhenru Zhang, Beichen Zhang, Runji Lin, Keming Lu, Bowen Yu, Dayiheng Liu, Jingren Zhou, and Junyang Lin. 2025. ProcessBench: Identifying Process Errors in Mathematical Reasoning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, pages 1009–1024. Association for Computational Linguistics.

Tianjun Zhong, Linyang He, and Nima Mesgarani. 2026. From Chains to DAGs: Probing the Graph Structure of Reasoning in LLMs. *CoRR*, abs/2601.17593.

Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V. Le, and Ed H. Chi. 2023. Least-to-Most Prompting Enables Complex Reasoning in Large Language models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.

Jiaru Zou, Ling Yang, Jingwen Gu, Jiahao Qiu, Ke Shen, Jingrui He, and Mengdi Wang. 2025. ReasonFlux-PRM: Trajectory-Aware PRMs for Long Chain-of-Thought Reasoning in LLMs. *CoRR*, abs/2506.18896.

A ReasoningFlow annotation guide

This section includes annotation guides for nodes and edges. For each subtype of nodes and edges, we provide one example to facilitate understanding of the presented labels due to spatial limitations. In the actual annotation guides for the manual annotators (Section 4.1) and LLM prompts, we provide the full set of examples, including an average of 2.05 examples per subtype.

Label	Description	Subtype	Example
`Context`	**Context** includes all *user-provided* texts. Some examples include the problem statement, retrieved documents, and tool responses.		
`Planning`	**Planning** introduces the content of the following nodes. It can be both coarse, high-level direction that affects tens to hundreds of nodes, or highly local plans that only affect the next node. Planning node often includes phrases like (I need, I should, Let's, ...) indicating that the model should **act** in some way. However, it can also be a very short phrase that indicates the direction of the next node, such as *"Now, for $n \geq 2$:"* or *"Numerator:"*.	Introducing long-term directions and subgoals (global plans)	First, I need to remember what Gibbs free energy change, ΔG, represents.
		Phrases that initiate verification	I should also consider if there are any assumptions I'm making here.
		Phrases that initiate backtracking (alternative solutions)	Alternatively, maybe I can think in terms of permutations with restrictions.
		Introducing the direction of the next node (local plans)	Let me calculate the numerator:
		Indicating the Conclusion node	Final Answer:
`Fact`	**Fact** contains external, parametric knowledge that is independent from the information provided in the context. A fact node must satisfy two criteria: 1. The information should not be given in any of the previous nodes including contexts. If so, it should be considered as restatement nodes. 2. It must not include nor reference information (numeric values, propositional content, context-specific constraints, ...) from the previous steps. If so, it should be considered as reasoning nodes.	Theorems, laws, and rules	E° cell = E° cathode + E° oxidation.
		Existing concepts and definitions	The Legendre symbol $\left(\frac{a}{p}\right)$ is defined as 1 if a is a quadratic residue modulo p and not divisible by p, and -1 otherwise.
		Values of well-known constants and unit conversions	I think A for nitrogen at 77 K is a known value.
		Factual knowledge	Now, hydrochloric acid (HCl) is a strong acid, which means it completely dissociates in water.
		Commonsense facts	2 ^10 is 1024.
`Reasoning`	**Reasoning** includes any of deductive/inductive/abductive inference. Reasoning nodes should not present novel facts; they must refer to previous nodes to derive the designated conclusion. Even if the derivation is trivial or obvious, it should still be categorized as Reasoning as long as it involves logical steps based on prior information.	Calculations	So, ΔG° = (-2150.4) - (-103.8) = -2150.4 + 103.8 = -2046.6 kJ/mol
		Logical reasoning	So, there are no real solutions to the inequality.
		Commonsense reasoning	When we increase the tax imposed on cigarettes, it will reduce the demand.
		Speculations	This logarithmic relationship suggests that the first digit changes in a way that is not periodic.
		Defining symbols and notations	Let the speed of the spacecraft be v.

Continued on next page

Label	Description	Subtype	Example
		Observations from examples	It seems that the provided numbers are all prime numbers.
`Restatement`	**Restatement** is when the model copies/paraphrases the preceding text (context/previous nodes). Therefore, the destination node should have content that can be directly entailed from the source node. If the node includes any new information that is not present in the source node, it should not be classified as Restatement. Typically, terms like 'as seen previously', 'already stated', 'the problem', strongly indicate the restatement nodes. However, it might simply restate previous steps without any of these auxiliary phrases.	Rephrasing the context nodes	The temperature is 25°C and the pressure is 1 atm.
		Rephrasing previous nodes	As before, this is approximately 13.928, which rounds to 14.
`Assumption`	**Assumption** is when a step is intentionally indicated as *not necessarily true*, but serves as a premise for the following steps. Therefore, Assumption defines a *scope* of nodes that are based on the assumption, where the validity of nodes depends on the validity of the assumption.	Assuming missing premises by commonsense/common practices	Since it's a swimming pool, I assume it's open,
		Assuming uncertain facts	Assuming the C-H stretching wavenumber in methane is around 3000 cm^{-1},
		Branching (switch-case)	If it were 4, ... Otherwise, if it were 6, ...
		Proof by contradiction	Let's suppose, for the sake of contradiction, that there exists an integer m such that $m^2 = 4n + 3$ for some integer n.
`Example`	**Example** is a specific instance of a general and abstract concept provided in the preceding steps. Examples are often unnecessary for the reasoning process, but rather helps illustrate the outline of the solution. Therefore, even if it is enumerating multiple objects, if it is directly solving the problem, it should be categorized as Reasoning. If a single example (each of the examples in a list) spans multiple nodes, label the first one as example, and the following nodes (derived from the first node) as accordingly.	Pattern induction by enumeration	Take $n = 1$:
		Non-exhaustive listings	Suppose $s = 13$, $s^2 = 169$, sum is $\frac{169-95}{2} = \frac{74}{2} = 37$, which is larger than 13.
		Rhetorical examples	For example, governments can provide services for verifying one's identity for financial transactions, preventing misuse.
`Reflection`	**Reflection** is a node that expresses *opinions* on the previous nodes. Reflections include both illogical emotions like *curiosity, uncertainty, and satisfaction*, and logical judgments like *correctness*. However, LLMs sometimes include subjective words during the reasoning process, as in "I think that (fact)" or "Wait, (fact)". Make sure the reflection nodes are not directly a part of the solution, i.e., removing the reflection node will make the reasoning trace still logically complete.	Meta-evaluation of a step	I'm clearly making a mistake here.
		Emotions and impressions	This is confusing.
		Rhetorical phrases	I'm not sure what it is off the top of my head.
		Filler words	I got this problem here.
		Reasoning on the applicability of Fact	However, Benford's law itself doesn't directly help in proving non-periodicity.

Continued on next page

Label	Description	Subtype	Example
`Conclusion`	This node includes the model's answer for the question (both final/intermediate). Final answer-based conclusion nodes are the ones that appear on the last attempt, which often includes phrases like **Final Answer**, **Answer:**, or similar expressions. However, conclusion nodes can contain *intermediate* answers. These conclusion nodes are usually the last step of an "attempt" to solve the problem, and are often followed by planning nodes that init verification. Exclude nodes that are scoped under assumptions. Make sure that the model is asserting the conclusion node as the potential final answer.	Intermediate conclusions	*(Question is asking for the time dilation factor)* Total time dilation factor: 0.9851 / 1.6667 ≈ 0.591.
		Final answers	Final answer is 5.

Table 4: Annotation guide for node labels.

Edge	Description	Subtype	Example
`infer`	This edge represents the basic premise-conclusion relationship. To apply this edge, both nodes should be asserted facts/propositions. There might be multiple premise nodes that are semantically equivalent. (1) If it is due to restatement, choose only the original node instead of the restatement. (2) Otherwise, choose the closest source node from the destination.	(Deductive) Premises → Conclusion	`fact`: *Sulferic acid is a strong acid,* ⇒ `reasoning`: *so it will fully dissociate, providing H+ ions.*
		(Math) Inputs and equations → Calculation results	`reasoning`: *r ^2 h = 2r ^3* ⇒ `reasoning`: *h = 2r*
		(Inductive) Observations → Inductive Hypothesis	`example`: *7 ^2=49, which is 4 * 12 + 1.* ⇒ `reasoning`: *Hmm, it seems like all these perfect squares are either 4n+0 or 4n+1.*
		Rationales for self-reflection	`reasoning`: *Wait, water cannot be liquid at 1000 K, 1 atm.* ⇒ `reflection`: *This seems off.*
`execute`	This label represents the relationship between a plan and its execution. The plan "introduces" what will happen in the subsequent reasoning step, and the subsequent nodes will follow the plan to perform actual reasoning. Note that this edge is mostly short-distanced, where it often connects nodes that are within distance 2. If there are parallel reasoning going on (e.g., by individually assessing two possibilities), this edge should connect the plan and the first nodes of these parallel reasoning chains. A step might be logically correct, but might not be relevant to the current goal. This edge provides the contextual information regarding what the step is doing right now, i.e., coherence of the reasoning step.	A plan and its implementation	`planning`: *Let me consider some examples:* ⇒ `example`: *- 0 ^2 = 0, which is 4×0 + 1.*

Continued on next page

Edge	Description	Subtype	Example
`restate`	This edge connects the Restatement node to the previous node that is restated. When there are multiple statements that include the same content, make sure that this edge points to the original node that is the earliest in the reasoning trace. Even if the node is not directly copying the content, restating a few facts or high-level information of the context can be considered as restatement.	Restating contexts	`context`: *What is the amplitude of gravitational waves produced by a binary black hole system with masses of 10 solar masses and 25 solar masses at a distance of 500 megaparsecs from the Earth?* ⇒ `restatement`: *Alright, I'm trying to figure out the amplitude of gravitational waves from a binary black hole system.*
		Restating intermediate nodes	`assumption`: *Let's suppose, for the sake of contradiction, that there exists an integer m such that $m^2 = 4n + 3$ for some integer n.* ⇒ `restatement`: *I previously assumed that an integer m satisfies $m^2 = 4n + 3$.*
`elaborate-fact`	This edge connects general facts to more specific facts. This edge is often used when the LLM generates consequent nodes of facts, and the following nodes describe the details of the initial fact. Note that this edge is usually short-distanced, where it is often connecting nodes that are within distance 2. Even if the two facts are semantically related, if they are not introduced within the same context, they should be not connected.'	A fact and linked facts	`fact`: *Okay, so I remember that the Arrhenius equation relates these quantities.* ⇒ `fact`: *The equation is k = A * e ^(-Ea/(RT)),*
`exemplify`	This edge label applies to Example nodes, connecting the example to what it exemplifies. When both nodes are trivia-like facts, it should be rather annotated as fact-detail.	Enumeration	`planning`: *First, maybe I should think about what perfect squares look like when divided by 4.* ⇒ `example`: *- 0 ^2 = 0, which is 4×0 + 0.*
		Conceptual examples	`planning`: *Maybe the problem is considering some other reaction or there's additional information I need to account for.* ⇒ `example`: *Perhaps the presense of sulfuric acid allows for copper to be oxidized directly, or there's a coupled reaction happening that I'm not accounting for.*
		Demonstrative examples	`reasoning`: *Therefore, the valid range is 0 < c < 9.* ⇒ `example`: *Take c=4, which is between 0 and 9.*

Continued on next page

Edge	Description	Subtype	Example
`proceed`	This is the default label that connects a plan to any nodes which the content triggers the planning. Typical examples include: - deciding high-level plans based on known facts - indicating how to solve (simplify, substitute, calculate) a certain equation at a low level - multiple planning/assumption nodes forming a linear sequence (first, second, .../ next, then, ...) verify, decompose, and backtrack edges are exceptions; they should be annotated with higher priority than this label. A rule of thumb for determining source nodes that are not planning: if a plan leads to a reasoning node via "execute" edge, the relevant source nodes of that reasoning node (annotated as infer/execute) should be connected to the plan via this edge. A rule of thumb for determining planning source nodes: the planning must connect to the very next planning node in a linear chain.	Steps that motivate the next plan/assumption → a plan/assumption	`reasoning`: *I don't see any patterns in the sequence.* ⇒ `assumption`: *Let's assume that the sequence is not periodic.*
		The previous plan's conclusion → next plan	`reasoning`: *So, M_c = 250 ^(3/5) / 35 ^(1/5).* ⇒ `planning`: *I need to calculate that.*
		Linear progression of planning	`planning`: *Left side:* ⇒ `planning`: *Right side:*
		Mutually exclusive assumptions	`assumption`: *Let's consider an even number: let's say 2k, where k is an integer.* ⇒ `assumption`: *Now, consider an odd number: let's say 2k+1, where k is an integer.*
		The reason a plan is not working → alternative plan	`reasoning`: *But I don't have a specific frequency given in the problem.* ⇒ `planning`: *Maybe I need to use the frequency at a certain point in the waveform, like the frequency at merger.*
		Steps that motivate self verification → Planning node that initializes verification	`reflection`: *This seems off.* ⇒ `planning`: *Maybe there's something missing.*
`verify`	This edge specifically labels the relationship between a reasoning-like node and a Planning node, initiating the verification process. Typical destination nodes include phrases like "Let's verify this", "I should check if there is any error".	Initiating verification	`reasoning`: *So, a + b = 7/2.* ⇒ `planning`: *But let me double-check.*
`decompose`	This edge connects a plan and a subplan. Subplans decompose the high-level plan into smaller, more straightforward goals. If the plans can be executed in parallel, edges should connect the high-level plan to all subplans; if the subplans are meant to execute sequentially, edges should connect to the first plan only. This is the most standard relationship between plans, so if there is a clear semantic relationship between two plans but no strong evidence for proceed or backtrack, this edge should be used.	Decomposition of coarser plans into finer subplans	`planning`: *Let's compute this equation step by step.* ⇒ `planning`: *First, computing 93.51 × 123.7:*
		Dividing a coarse assumption into fine-grained ones	`assumption`: *Let's assume that n ^2 is either 0 or 1 modulo 4.* ⇒ `assumption`: *Case 1: n ^2 ≡ 0 (mod 4)*
		Paraphrasing of a planning node due to formatting reasons (should only select when two nodes are adjacent)	`planning`: *Let's consider the case for x=5.* ⇒ `planning`: *### Case 1: x=5*
`backtrack`	This edge denotes when the model is suggesting an alternative plan from the previous one. The subsequent nodes often include phrases that indicate backtracking, e.g., "Alternatively", "Let me think in other direction". When there are multiple alternative Planning steps chained linearly, one should only link the immediately preceding one and not connect between all possible pairs.	Alternative plans (backtracking)	`planning`: *Also, I should consider if there are any other factors that might affect the pressure, like temperature or the exact value of gravity,* ⇒ `planning`: *Another thing to think about is whether the pool is open to the atmosphere or not.*

Continued on next page

Edge	Description	Subtype	Example
`positive`	Reflection edges connect a planning/reasoning-like statement to a reflection node that contains personal views and emotions about the statement. This edge connects a reasoning node (mostly Reasoning, Fact, Assumption) and a reflection node that affirms it. If the affirmation is based on the semantic identity of previous and current steps, e.g., connecting the conclusion of the positive self-verification to the verification goal, it should be annotated as support. If the reflection node compares/contrasts multiple nodes (e.g., "This seems consistent with previous results"), create edges for both nodes that are being compared.	Affirmation of previous nodes	`conclusion`: *H2SO4 + Ca(OH)2 → CaSO4 + 2 H2O* ⇒ `reflection`: *I think that's it.*
`negative`	Reflection edges connect a planning/reasoning-like statement to a reflection node that contains personal views and emotions about the statement. This edge connects a reasoning node and a node that denies it. This is mostly caused by reflection statements like "This seems incorrect" or reflecting without explicit correction edge, such as "Therefore, it is wrong". When it explicitly corrects the previous node, it should be annotated as attack. If the reflection node compares/contrasts multiple nodes (e.g., "This seems off given the previous results"), create edges for both nodes that are being compared.	Negative evaluation of previous reasoning results	`reasoning`: *- H: 2 + 2*2 = 6?* ⇒ `reflection`: *Wait, no.*
		Negative judgment on the applicability of a fact	`fact`: *Van der waals forces are weak intermolecular forces that arise from induced dipoles in molecules.* ⇒ `reflection`: *However, I think that it will be negligible in this case.*
		Negative judgment on whether the plan will work or not	`planning`: *We can try considering Van der waals forces.* ⇒ `reflection`: *However, I think that it will be negligible in this case.*
`uncertain`	Reflection edges connect a planning/reasoning-like statement (source) to a reflection node (dest) that contains personal views and emotions about the statement. This edge represents all reflections that are not explicitly positive or negative. Typical use cases are reflection of uncertainty ("I am not sure/certain"), confusion ("I am confused/lost"), lack of knowledge ("I don't know"), lack of confidence ("I am a bit rusty"), and anomaly ("This seems weird"). If the reflection node compares/contrasts multiple nodes (e.g., "This is different from the previous result, which confuses me"), create edges for both nodes that are being compared.	Uncertainty	`planning`: *But wait, Is this formula correct?* ⇒ `reflection`: *I'm not entirely sure.*
		Confusion	`reasoning`: *t = 550 nm / 1.5 = 366.67 nm.* ⇒ `reflection`: *This is different from the previous results.*
		Lack of knowledge/ability	`reasoning`: *t = 550 nm / 1.5 = 366.67 nm.* ⇒ `reflection`: *But again,. without a clear theoretical basis, it's hard to be confident.*
		Lack of confidence; too difficult	`reasoning`: *but again, that's for reflections, not necessarily for transmission through a material.* ⇒ `reflection`: *This is tricky.*
		Exclamation	`reasoning`: *so total Ea = 94,147.736 + 691.33 ≈ 94,839 J/mol.* ⇒ `reflection`: *That seems high.*

Continued on next page

Edge	Description	Subtype	Example
`support`	While other edges are focused on the logical premise and the reasoner's intent, Validate edges denote whether two pieces of information are consistent or not. This edge label applies to reasoning nodes and conclusion nodes that reinforce previous nodes by stating the same thing. It distinguishes from restate because it is not blindly copying the previous statement, but comparing two pieces of information that were derived independently. It also differs from positive, because support annotates propositional equivalence while positive only annotates positive sentiment. Finally, 'Conclusion' nodes must connect to all previous 'conclusion' nodes that address the same question, via support or attack.	Positive self-verification conclusion	`reasoning`: *Now, ln(A/k) = ln(5e13 / 2.5e-3) = ln(2e16) ≈ 38.279, as I had before.* ⇒ `reasoning`: *So, I use 38.279, which is close enough.*
		Conclusion node and preceding consistent conclusions	`conclusion`: *H2SO4 + Ca(OH)2 → CaSO4 + 2 H2O* ⇒ `conclusion`: *H2SO4 + Ca(OH)2 → CaSO4 + 2 H2O*
		The context is a proposition, and the conclusion is its affirmation	`context`: *Should cigarettes be banned for all ages?* ⇒ `conclusion`: *As my final opinion, I believe that all cigarettes should be banned for all ages.*
`attack`	While other edges are focused on the logical premise and the reasoner's intent, Validate edges denote whether two pieces of information are consistent or not. This edge label applies to Restatement nodes and Conclusion nodes, connecting the original statement to the restatement. It differs from negative, because attack annotates propositional equivalence, while negative only annotates positive sentiment. Finally, 'Conclusion' nodes must connect to all previous 'conclusion' nodes that address the same question, via support or attack.	Proof by contradiction	`assumption`: *Let's suppose, for the sake of contradiction, that there exists an integer m suc that m ^2 = 4n + 3 for some integer n.* ⇒ `reasoning`: *it must be that no integer m exists such that m ^2 = 4n + 3.*
		Explicit correction	`reasoning`: *H: 2 + 2*2 = 6?* ⇒ `reasoning`: *H2SO4 has 2 H, and Ca(OH)2 has 2*(1) = 2 H, so total 4 H.*
		A claim and preceding claims that are logically inconsistent/mutually exclusive	`conclusion`: *The answer is Wednesday.* ⇒ `conclusion`: *The final answer is Tuesday.*
		The context is a proposition, and the conclusion is its negation	`context`: *Should cigarettes be banned for all ages?* ⇒ `conclusion`: *To conclude, I think that cigarettes should be allowed to adults.*

Table 5: Annotation guide for edge labels.

B Extended related works

B.1 LLM reasoning

Entailment graph and verification. Reasoning was traditionally viewed as combining existing pieces of knowledge to deduce new facts. Dalvi et al. (2021) introduced *entailment graph* structures that connect logical premises to their conclusions. Under this formulation, reasoning becomes tree expansion, where one recursively adds intermediate conclusions from the given leaf nodes (facts) until the desired conclusion is reached. The definition of logical premises is straightforward in some tasks. In arithmetic word problems (Cobbe et al., 2021), the premises are where the numbers used in the current step are first derived (Li et al., 2023); in logical reasoning tasks with corresponding formal logic representation, *natural deduction* reveals the logical premise to deduce the current step (Han et al., 2024).

More recent works (Ling et al., 2023; Mukherjee et al., 2025) developed a general definition of premises based on the *stepwise evaluation* task (judging whether a step is correct or erroneous). In this setting, the premise set should be *complete*, providing sufficient information to determine whether the step is correct or erroneous, and *minimal*, so that removing any premises makes the set incomplete. Premise selection based on such criteria leads to accurate and efficient step verification by removing distractors. ReasoningFlow directly adopts this definition as one of the main design principles, so that the set of connected previous nodes forms a *minimal complete set* as defined above.

However, these works are solely limited to entailment relationships, ignoring diverse discourse relations present in LRM traces beyond entailment. Even before LRMs, behaviors like plan decomposition and backtracking were identified as critical components of LLM reasoning (Zhou et al., 2023; Yao et al., 2023). While these behaviors clearly differ from deductive reasoning, existing works on entailment structure failed to capture these behaviors.

Behavioral analysis of LRMs. After DeepSeek-R1 gained popularity (Guo et al., 2025), immediate attempts to analyze their long and complex reasoning traces involved counting keywords like "Wait" (Chang et al., 2025) or using few-shot LLMs to classify reasoning behaviors (Gandhi et al., 2025). However, these approaches focused on the *existence* of such behaviors, rather than identifying the underlying discourse structure.

DeepSeek-R1 Thoughtology (Marjanovic et al., 2026) identified the global structure of reasoning traces in four stages. The framework states that LRMs first tend to restate the problem in their own language (*Problem definition*), derive an initial solution (*Bloom*), try recomputation or alternative approaches to verify the initial solution (*Reconstruction*), and decide the final answer (*Final decision*). Their proposed structures are semi-linear; the four stages occur linearly, while the reconstruction stages repeat multiple times. While this observation broadly applies to different reasoning models, it fails to address *nested* behaviors of different granularities (e.g., LLM includes a short self-verification during the Bloom stage). Furthermore, their annotations are based on paragraphs, which disallows fine-grained intent analysis and verification.

LCoT2Tree (Jiang et al., 2025) and ReJump (Zeng et al., 2025b) introduce a tree structure to annotate additional information about where verification and backtracking happen. The Bloom stage in Marjanovic et al. (2026) is expressed as a sequence of paragraphs, and any backtracking or verification attempt is shown as branching out from one of the paragraphs. This structure can easily express arbitrary backtracking and verification, which is prominent in tasks involving heavy depth-first search like Game of 24 (e.g., *Use 2,3,5,6 and arithmetic operations to make 24*). While they can express non-entailment relationships and non-linear reasoning structures, they cannot express fine-grained reasoning behaviors, like self-reflection and assumption, which require more expressive node and edge labels.

Thought anchors (Bogdan et al., 2025) offer *sentence*-level analyses of reasoning trace structures. They propose eight functional roles of sentences, where some of them directly correspond with ReasoningFlow labels (*Plan generation* and `Planning`, *Fact retrieval* and `Fact`, *Deduction/Active computation* and `Reasoning`, *Example testing* and `Example`, *Final answer emission* and `Conclusion`). However, their node annotations focus on *global* roles. For instance, steps within global verification (*Reconstruction* stage) are always annotated as *Self-checking* in Thought Anchors, but vary by their sentence-level semantics in ReasoningFlow. This leads to discrepancies when applying the structures to *downstream tasks*.

For example, since we cannot evaluate the validity of non-reasoning nodes like `Planning`, we must further distinguish whether a `Self-checking` sentence contains reasoning content or not.

Edge annotation is also significantly different between Thought Anchors and ReasoningFlow. Thought Anchor's edge annotations were purely mechanistic and did not distinguish between different discourse relations. ReasoningFlow offers a fine-grained taxonomy of the relation between two steps. We also find that annotated edges between the two methods show a significant gap; refer to the main text (Section 8) and Appendix H.

Finally, we compare the current version of ReasoningFlow to the `preview` version (Lee et al., 2025). The node labels have been updated in several ways: reflective statements that reason about a previous step are now classified as `Reflection` rather than `Reasoning`, and `Conclusion` has been introduced to distinguish the final answer of each attempt from other nodes. For the edge labels, most have been renamed for consistency. We also introduce validate edges (`support`, `attack`) to capture long-range agreements and disagreements between segments, and merge incoming edges of `Planning` from both planning and non-planning nodes into a single label (`proceed`).

B.2 Adjacent fields

While ReasoningFlow's core design is mainly influenced by the LLM reasoning literature, it is also inspired by diverse fields, including computational linguistics, formal logic, and cognitive science.

B.2.1 Discourse parsing

Rhetorical Structure Theory (RST) RST was first proposed by Mann and Thompson (1988), aiming to discover the structural organization of long coherent texts. RST constructs a discourse tree of clause spans, where *satellite* spans are recursively adjoined to the *nucleus* span based on the rhetorical intent (*Why did the speaker add this clause?*). This theory led to the development of the RST-DT dataset (Carlson et al., 2001), annotating 17 different semantic relations between adjacent spans.

While ReasoningFlow and RST-DT both annotate discourse structures for fine-grained units, they exhibit significant structural differences. RST-DT annotates projective tree structures without crossing edges, while ReasoningFlow uses a directed acyclic graph structure that allows arbitrarily crossing edges. We find that projectivity constraints cannot coexist with the core design principle of ReasoningFlow (edges should capture minimal but *sufficient* context of a step), as approximately 42.6% of edges should be removed to ensure projectivity in the ReasoningFlow dataset.

In the label set, the most noticeable difference is the granularity of *inference* behaviors. `infer` in ReasoningFlow can correspond to many types in RST-DT: `background`, `cause`, `explanation`, `summary`, and `condition`. Making `infer` more general is a design choice backed by two reasons. First, the five labels do not fully cover the corner cases in `infer` edge. For instance, the third subtype of `infer` (Table 5) annotates the inductive reasoning process that connects a single example to a general pattern; it does not directly correspond to `summary` (denotes many-to-one restatement) or `explanation` (examples are not a reason why the pattern is true). Second, dividing `infer` does not alter how to treat different labels in downstream tasks; i.e., the process for verifying the reasoning correctness of `summary` and `explanation` pairs will be identical.

However, for other reasoning-focused behaviors than inference, ReasoningFlow is significantly more expressive. First, while ReasoningFlow includes many long-distance relations that connect different stages of reasoning (`backtrack`, `support`, `attack`), RST lacks these long-distance relations due to projectivity constraints. Second, while RST annotates attitude to the nucleus via a single *evaluation* edge, ReasoningFlow also annotates the sentiment (`positive`, `uncertain`, `negative`). Finally, RST covers relations between a plan and its implementation via `Manner-Means` or `Enablement`, but does not cover general cases as shown in the annotation guide for `execute`.

Penn Discourse TreeBank (PDTB) Another popular branch of discourse parsing is PDTB (Prasad et al., 2008), which annotated various discourse relations on top of the Penn TreeBank corpus of news articles from the Wall Street Journal. They annotate independent binary relations between clauses, unlike a fully connected tree in RST.

PDTB edges are defined using the semantics of *connectives* like "but" (`contrast`) or "Therefore" (`cause`). However, ReasoningFlow's edge definitions also include content-based relations that cannot be fully captured using connectives. For instance, `proceed` is purely annotated based on

whether the semantic content of the `Planning` step is motivated by the previous steps, and such a relation has no one-to-one correspondence with any connective in English.

B.2.2 Argumentation structure mining

In argumentative texts, argumentative components (claims, evidence) exhibit directed argumentative relations that indicate the degree of support of a component for another. Persuasive Essays (PE) corpus (Stab and Gurevych, 2017) is the representative resource for argumentative structure mining. PE annotates spans as *major claims*, *minor claims*, and *premises* along with *support* and *attack* relationships between these spans. While the exact label set varies within datasets, the general idea of identifying claims and premises with support and attack relations mostly remains consistent across argumentation mining literature (Lauscher et al., 2018; Habernal et al., 2024).

While argumentation mining frequently targets argumentative text like persuasive essays, academic papers, and legal judgments, a reasoning trace for objective tasks (e.g., math) can also be viewed as a type of argumentation. In this analogy, `major claims` corresponds to the final answer (`Conclusion`), where `Fact` and `Reasoning` roughly correspond to leaf evidence (base facts) and minor claims derived from evidence, respectively. In terms of edges, *support* relation in argumentation mining corresponds to `infer` and `support`[1].

Compared to argumentation mining, ReasoningFlow includes more fine-grained discourse relations for diverse reasoning behaviors. We claim that ReasoningFlow's expressive label sets possess the potential to expand the scope of argumentation mining. For instance, existing works seldom identify rare argumentation strategies like hypothetical reasoning (e.g., *reductio ad absurdum*) or analogical reasoning (Walton et al., 2008); ReasoningFlow's `Assumption` and `Example` nodes naturally distinguishes these strategies from common deductive argumentation.

B.2.3 Natural deduction

Natural deduction is a calculus for formal logic that involves *natural* inference rules that align with human instincts, such as *modus ponens*, *modus tollens*, and hypothetical syllogism.

Natural deduction systems can be classified by their reasoning directions: bottom-up and top-down. Bottom-up systems combine base facts to repeatedly deduce new conclusions until the conclusion is reached (Lifschitz, 2019; Tafjord et al., 2021; Creswell et al., 2023). On the other hand, top-down systems start from the desired conclusion (goal) and recursively decompose it into subgoals until verified (Wielemaker, 2003; Kazemi et al., 2023; Lee and Hwang, 2025). In ReasoningFlow, bottom-up deductive reasoning and top-down decomposition/planning are both captured via `infer` and `decompose` edges, respectively.

Furthermore, assumptions play a significant role in natural deduction, which motivated the `Assumption` nodes in ReasoningFlow. In natural deduction, a *subproof* is a nested block of reasoning where a statement is temporarily assumed, and the block is discharged when the expected conclusion is proved. It is frequently utilized by inference rules like conditional proofs (To prove $p \rightarrow q$, assume p and show q), or-elimination (Given $p \vee q$, prove $p \rightarrow r$ and $q \rightarrow r$ to show r), and proof-by-contradiction (To prove a, show that $\neg a$ leads to a contradiction).

B.2.4 Cognitive science

Whether LLMs are *meant* to resemble human cognitive behaviors during the Chain-of-thoughts reasoning is a debatable question (Bao et al., 2025; Chen et al., 2025; Hao et al., 2024). However, it is widely accepted that cognitive science provides a useful guide for understanding LLM reasoning (Zhang et al., 2026; Liu et al., 2025). In this section, we use Kargupta et al. (2025)'s exhaustive taxonomy of cognitive concepts related to human and LLM reasoning *operations*, and their relation to ReasoningFlow.

Reasoning Navigation. The three core operations for navigating the reasoning process are `Forward chaining`, `Backward chaining`, and `Backtracking`. Forward chaining recursively combines known facts towards the final goal, while backward chaining decomposes goals into prerequisites. Similar to bottom-up and top-down inference in natural deduction, forward chaining and backward chaining directly correspond to `infer` and `decompose`. Finally, backtracking is the ability to revisit and correct prior reasoning paths, of-

[1] Seldomly, `positive` edge (affirmation of previous nodes) if the destination `Reflection` node includes a specific reason – as in *"I think that's it, because the boiling point of Benzo derivatives are typically around 180°C to 330°C."* – instead of only having filler phrases like *"I think that's it."*

ten implemented as a depth-first search that returns to the previous decision point to explore alternatives. In ReasoningFlow, backtracking is denoted by `backtrack` or `Assumption` connected via `proceed`, which directly connects the failed strategy/assumptions to a new alternative.

Verification. `Verification` evaluates whether the reasoning steps are consistent, plausible, and coherent with the provided facts or the world state, which is captured by `verify`. While there are orthogonal criteria for evaluating the reasoning steps, such as coherence or utility (Lee and Hockenmaier, 2025), we do not further distinguish the detailed intent because most self-verification processes in LRMs are focused on logical validity.

Modifying knowledge representations. Kargupta et al. (2025) also proposes a wide terminology for operations that derive new information from existing ones.

`Pattern recognition` involves identifying recurring *templates*, or applying a general idea to a problem-specific knowledge. In reality, this is captured by determining which rule or idea to apply to the current pool of knowledge. It is captured by ReasoningFlow's `proceed` that derives a general plan from previous steps. `Abstraction` is when one detects abstract, generalizable patterns from specific instances, also known as inductive reasoning. In ReasoningFlow, this corresponds to when multiple `Example` (that `exemplify` a same concept) collectively deduce a single conclusion. ReasoningFlow does not distinguish between abstraction and ordinary deductive reasoning at the edge level because they can be identified by node labels.

`Representational restructuring` involves reformulating the goal to obtain new insights that lead to a better solution. Kargupta et al. (2025) applies the widest possible definition of restructuring for analyzing reasoning traces in the focus of *cognitive* behaviors, ranging from decomposing objective subgoals from a subjective problem (e.g., changing *Which cellphone is better?* to *Which cellphone has more memory?* to restructuring the given equation for a better solution (e.g., partial fraction decomposition). However, at the discourse structure level, we claim that these instances fundamentally differ in their semantics, e.g., explicitness or logical equivalence. Hence, we apply the most appropriate edge labels for each of the cases, e.g., `decompose` for the former example and `infer` for the latter.

C Automatic annotation (§4.2) details

C.1 Implementation details

Node segmentation. When asking LLMs to segment an entire reasoning trace into nodes, we have frequently observed two failure modes: (1) segmented nodes do not exactly match the original trace, and (2) some segments have excessive length (10,000+ characters). To address these issues, we first apply rule-based segmentation and then use LLMs to further segment them into atomic units.

We begin the segmentation process by rule-based detection of paragraphs. Instead of using all double newlines `\n\n` as paragraph delimiters for reasoning traces, we split only when the preceding paragraph ends with a period, and the next paragraph starts with a capital letter. This is to ensure that a single semantic unit (e.g., equation series in a single `begin...end` block) is not mistakenly segmented into two parts.

After the initial coarse segmentation, we leverage LLMs to further segment the chunks into ReasoningFlow nodes. If any segments are not perfectly aligned with the original string (often whitespaces or TEX symbols), we use dynamic programming to find the optimal alignment between the remaining segments, maximizing the total longest common subsequence length across all unmatched components.

Node classification. Instead of classifying the nodes independently, we annotate all nodes simultaneously in a single inference, based on our preliminary results. We provide node definitions and examples for each subtype, as shown in Table 4.

Post-hoc annotation of Conclusion nodes. Every reasoning trace must contain at least one `Conclusion` node by definition; however, we find that initial annotation occasionally omits one even if instructed. To prevent this, we adopt a two-stage procedure where the LLMs first classify all nodes with labels excluding `Conclusion`, and then identify which should be labeled as `Conclusion`. This design ensures that every reasoning trace contains at least one `Conclusion` node.

Edge detection and classification. We consider three possible implementations for edge detection: (1) annotating all edges simultaneously, (2) annotating incoming edges per node, and (3) annotating all node pairs individually (*dyadic*). Following Mukherjee et al. (2025), we adopt (2) for its

Model	F1 score		Krippendorff's α	
	NC	EDC	NC	EDC
GPT-5-mini	0.792	0.425	0.741	0.444
GPT-5.1	0.833	0.574	0.761	0.604
Gemini-2.5-Flash	0.811	0.535	0.749	0.576
Gemini-3-Flash	0.865	0.583	0.820	0.607
Gemini-3.1-Pro	0.859	0.646	0.812	0.677

Table 6: F1 scores and inter-annotator agreement (Krippendorff's α) measured between human annotators and LLM models with ground-truth segmentation. F1 scores are macro-averaged for each reasoning trace. Gemini-3-Flash achieves the best score in node classification (NC), while Gemini-3-Pro exhibits strong edge detection/classification capability (EDC).

balance of annotation quality and computational efficiency. For each node, we prompt LLMs to identify the minimal complete set of predecessors along with their corresponding edge labels.

ReasoningFlow edges are type-dependent: each node type admits only a restricted subset of incoming edge labels. For example, `positive` can only flow into `Reflection` by definition. We therefore provide the set of permitted edge labels for each node type when querying for predecessors, reducing label ambiguity.

The prompts for LLM annotation in all stages are presented in Appendix I.

C.2 Automatic annotation performance

Model performance. To select the LLM to use for automatic annotation of ReasoningFlow, we compare various LLMs using the manually annotated set in Section 4.1.

Table 6 reports the F1 scores for node classification (NC) and edge detection/classification (EDC)[2], alongside inter-annotator agreement metrics for direct comparison with human-human agreement in Table 3. For Node Classification, Gemini-3-Flash demonstrates the best performance overall, while for Edge Detection/Classification, Gemini-3-Pro exhibits strong performance, being the only model to achieve $\alpha \geq 0.67$ for EDC. All results are performed with a single run of greedy decoding.

Following common practices in discourse parsing literature (Marcu, 2000; Morey et al., 2017), we compare models using the same ground-truth segmentation as manual annotations. In the following section, we show that automatic segmentation generates different boundaries from human predictions, but the final discourse structures are topologically compatible in most cases.

Confusion matrix analysis. Figure 9 shows the confusion matrix of Gemini-3.1-Pro. Some common mistakes in node classification include classifying non-`Reasoning` nodes (`Fact`, `Reflection`, `Conclusion`) as `Reasoning`. The edge confusion matrix shows that misclassification of node labels directly affects the edge annotations, as the model confuses `elaborate-fact` (often $\rightarrow$ `Fact`), `negative` ($\rightarrow$ `Reflection`), and `attack` ($\rightarrow$ `Conclusion`) with reasoning nodes.

C.3 Automatic node segmentation

In this section, we explore the effects of errors in automatic node segmentation. Due to the constraint that all nodes should be a sentence at longest, nearly all (95.6%) misalignments in the test set are one-to-many misalignments, i.e., one side does not segment a sentence, while the other further segments one. Among these one-to-many misalignments, LLMs prefer more fine-grained segmentation than humans, as the number of over-segmentations (64.2%) nearly doubles under-segmentations (35.8%).

How do these segmentation misalignments affect the downstream ReasoningFlow structure? We qualitatively find that these misalignments between humans and LLMs do not cause significant effects in the overall structure. Figure 10 compares automatic ReasoningFlow annotation over human and LLM segmentations; while the segmentation is misaligned, the topology of the overall graph remains semantically consistent. This is similar to discourse parsing, where a failure to segment two sibling leaf spans casts minimal effect on the global tree structure (Nguyen et al., 2021).

C.4 Manual verification

To ensure that the quality of automatically annotated ReasoningFlow data is acceptable, a single human annotator reviewed annotations for 30 randomly sampled reasoning traces (10 per dataset). Due to annotation resource constraints, we report *acceptability judgments* by comparing LLM annotations with human corrections (rather than annotating from scratch), following prior work (Mukherjee et al., 2025; Zeng et al., 2025b). Specifically, a human manually edited the LLM-produced annotation, updating the node/edge labels and

[2]For EDC F1 score, we do not treat `no-edge` as a positive label, unlike the calculation of Krippendorff's α.

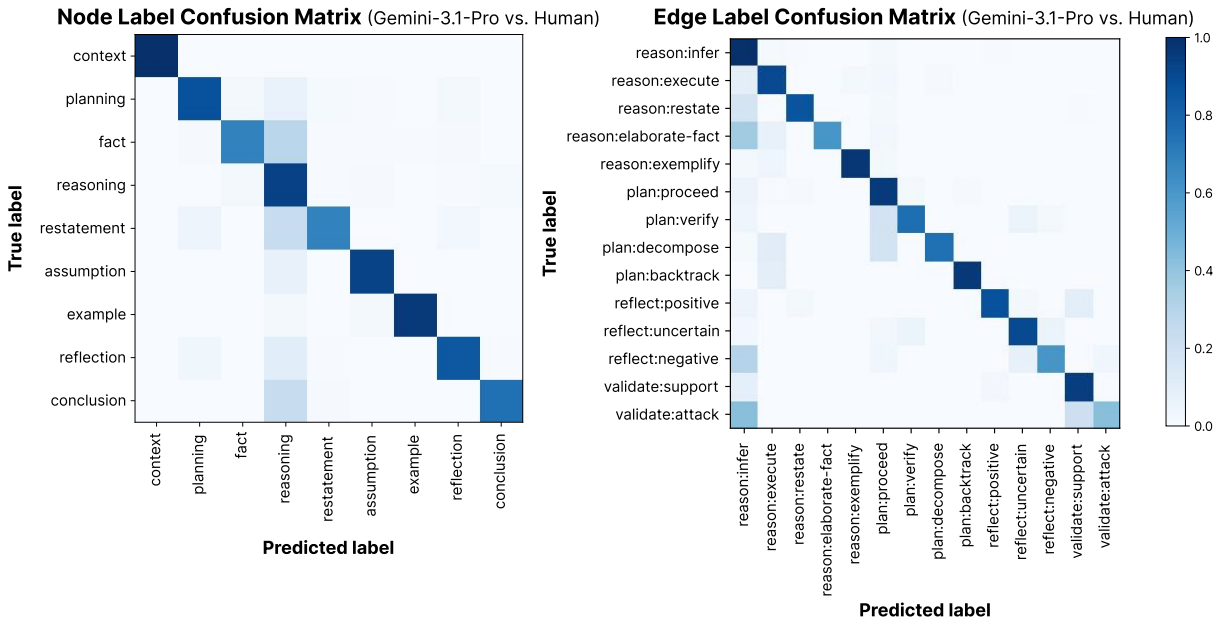


Figure 9: Confusion matrix (row-normalized) of Gemini-3.1-Pro annotations compared to human annotations.

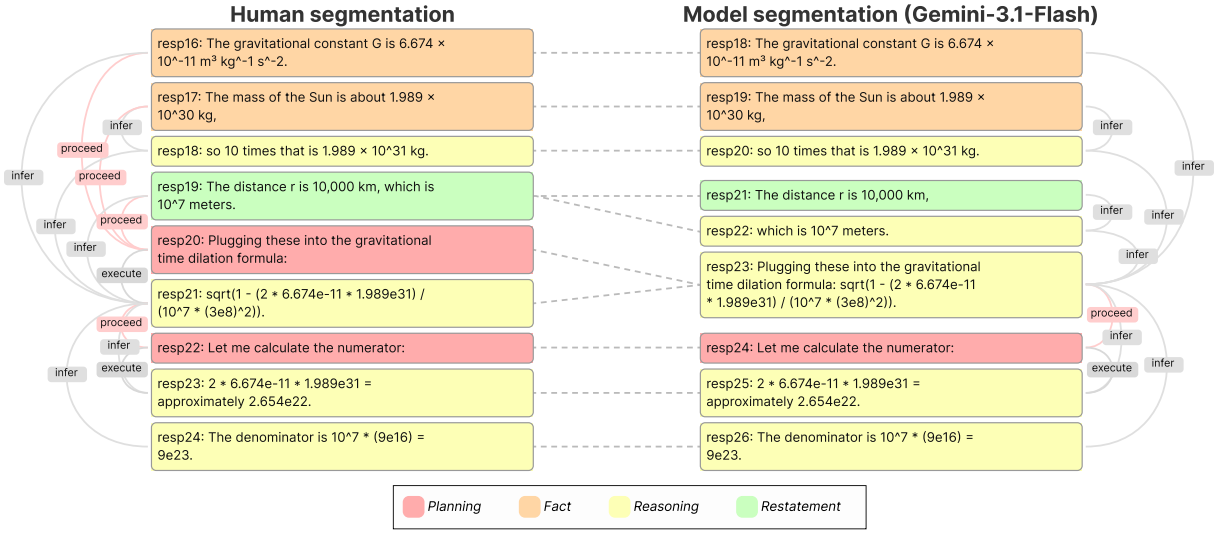


Figure 10: Example of a misalignment between human and LLM segmentation. Even if segmentations are misaligned, the overall structures remain semantically compatible. For instance, human segmentation's `resp21` node is topologically identical to model segmentation's `resp23` node, except `resp20`→`resp21` that connects the first and second half of the model's `node23` node.

adding/removing edges. Humans are instructed not to modify the segmentation.

The results indicate that LLM-based annotations are highly agreeable, achieving an F1 score of 0.909 for node classification (NC) and 0.869 for edge detection/classification (EDC). Note that evaluating with from-scratch human annotation instead of acceptability (Table 6) will likely reduce the inter-annotator agreement scores.

C.5 Computation cost

We annotate the ReasoningFlow dataset with the best LLM available (Gemini-3.1-Pro and Gemini-3-Flash) to achieve annotation accuracy. In the first pass, we annotate all traces with Gemini-3-Pro for edge detection/classification and Gemini-3-Flash for the remaining components. However, 289 of the first 1,260 annotations failed three times in a row due to the instability of the Gemini-3.1-Pro API. For these instances, we fell back to the second pass, only using Gemini-3-Flash for all annotation tasks.

At the time of the experiment, Gemini-3-Pro API cost $2.00/1M input tokens and $12.00/1M output tokens; Gemini-3-Flash cost $0.50/1M input tokens and $3.00/1M output tokens. The first pass (Pro and Flash) cost a total of $1,032.98 with 442M input tokens and 12.3M output tokens (both models combined), and the second pass (Flash only) cost $1,571.59 with 754M input tokens and 5.2M output tokens.

D Node quality annotation (§4.3) details

D.1 Stepwise evaluation with PARC

To evaluate the validity (logical correctness) of each node, we employ the LLM-as-a-judge approach (Gu et al., 2024). We prompt the LLM with a step and its context (question, previous steps), and make it reason about whether the given step is logically correct or incorrect based on the context. As ReasoningFlow includes extremely long reasoning traces, it is not feasible to provide *all* previous steps as the evaluation context. Instead, we prune the context using the ReasoningFlow graph by selecting all preceding nodes within a graph distance of 2. As explored in non-reasoning models, context pruning improves verification performance by removing irrelevant context, while reducing the inference cost (Ling et al., 2023; Mukherjee et al., 2025; Lee and Hockenmaier, 2025).

Another benefit of using graph structures for stepwise evaluation is the distinction between direct and propagated errors. Mukherjee et al. (2025) defines direct errors as steps that are erroneous but their premises are all correct, and propagated errors as steps that are correctly inferred from erroneous previous steps. As it is challenging to decide whether the propagated error should be considered an error (Lightman et al., 2024; Zheng et al., 2025), the ternary labels (correct, direct error, propagated error) significantly reduce ambiguity.

True error?	Count
Yes	94 (61.4%)
Somewhat	29 (19.0%)
No	30 (19.6%)
Total	153 (100%)

Table 7: Manual verification of PARC error detection accuracy, tested for 50 files with PARC-detected errors.

We also perform manual verification of PARC results, judging whether the PARC-predicted errors are true errors (i.e., *precision*). Table 7 shows that most of the errors (80.4%) are clearly or somewhat errors, indicating that the combination of PARC and ReasoningFlow is highly effective when detecting errors in LRM traces. While not directly comparable, the precision is significantly higher than the reported precision in DeltaBench (He et al., 2025) for detecting errors in LRM traces (GPT-4-Turbo 37.4%).

Error detection with PRMs. Process Reward Models (PRMs) (Uesato et al., 2022; Lightman et al., 2024) are LLM-based classifiers specifically trained to predict whether the given step is correct or not. However, we do not apply PRMs for several reasons. First, state-of-the-art PRMs like Qwen2.5-Math-PRM-72B (Zhang et al., 2025) are not trained to evaluate errors in long LRM traces. Consequently, 24-51% of their error predictions in ReasoningFlow are `Planning` or `Reflection`, which cannot be erroneous by definition, indicating that PRMs are not directly applicable for evaluating LRM traces. Furthermore, PRMs often have limited context length (e.g., Qwen2.5-Math-PRM can take 4096 tokens), making it infeasible for detecting errors beyond that limit. While recent works explore PRMs specifically trained for reasoning models (Zou et al., 2025; Xie et al., 2026), they have not been verified against LRM-focused stepwise evaluation benchmarks (e.g., He et al. (2025))

Arg. Match?	Count
Yes	338 (84.3%)
Somewhat	52 (13.0%)
No	11 (2.7%)
Total	401 (100%)

Table 8: Manual verification of argument alignment between ReasoningFlow's Reasoning nodes and AQR-30k's human-annotated arguments on the ArgKP subset.

or have not released the PRM checkpoint.

D.2 Argument quality scoring with AQR

To estimate the quality of reasoning nodes in the ArgKP dataset (Bar-Haim et al., 2020), we use AQR-30k (Gretz et al., 2020). AQR-30k includes crowdsourced arguments with their scores for 71 debate topics, which include all 24 topics in ArgKP. They propose WA (Weighted Average) scores to reduce the contributions of unreliable score annotators, which we use as the argument score.

We observed that AQR-30k includes logically equivalent arguments with significantly different scores. For instance, the two arguments against capital punishment: *"Capital punishment is against god's will."* and *"Only god decides who lives and who dies; capital punishment should not exist."* are almost equivalent, but the WA-scores assigned to them are 0.42 and 1.0, respectively. To reduce variance, we prompt Gemini-3-Flash to map a ReasoningFlow node to multiple AQR-30k arguments that are logically equivalent, and take the average score of the mapped AQR-30k arguments as the node quality. Table 8 shows the manual verification results of the matching between randomly sampled 50 nodes and 401 arguments, proving that such LLM-based matching is highly precise.

E ReasoningFlow statistics (§5) details

Label distributions. Figure 11 shows the node and edge label distribution in ReasoningFlow. Since most tasks are reasoning-focused, `Reasoning` occupies the largest portion of the nodes. While `Assumption` and `Example` are rare (2.8% and 1.1%, respectively), they carry unique semantics for identifying reasoning behaviors like assumption scoping and inductive reasoning and cannot be neglected. For the edges, `infer` is the most common label as it is the primary label for connecting the most common `Reasoning` nodes.

Triplet	Loading
PC1 (41.0%)	
`Context` – `infer` → `Reasoning`	-0.321
`Planning` – `proceed` → `Planning`	+0.311
`Reasoning` – `proceed` → `Planning`	-0.277
PC2 (20.2%)	
`Context` – `infer` → `Reasoning`	+0.395
`Conclusion` – `support` → `Conclusion`	+0.377
`Reasoning` – `infer` → `Reasoning`	-0.364

Table 9: Top-3 PCA loadings for first and second principal components in triplet analysis (Figure 3(a)).

Triplet analysis details To faithfully analyze triplet distribution using PCA, we use Hellinger PCA, i.e., PCA on vectors of $\sqrt{\text{probability}}$ (Lebret and Collobert, 2014). Taking the square root of probability improves fairness in distance for sparse probabilities. For instance, distance metric $|t_1 - t_2|$ treats the difference $.5 - .51$ identical to $.01 - .02$, while $|\sqrt{t_1} - \sqrt{t_2}|$ assigns more weight to the latter. Since ReasoningFlow has a very sparse triplet distribution, we find Hellinger PCA a more appropriate approach than PCA on the raw probability vectors.

Regarding Figure 3(a)'s two principal components (PC), the top 3 triplets with the largest loadings (i.e., $v_{\{1,2\}}^{\top} e_i$ are presented in Table 9. Compared to AIME and GPQA (`-PC1`), ArgKP traces (`+PC1`) are characterized by chained `Planning` nodes that enumerate minor claims (*"First, evidence supporting flag burning should be allowed due to freedom of speech: (evidence) Second, ..."*), leading to frequent `proceed` edges. GPQA traces (`+PC2`) frequently reference `Context` nodes that include important information like conditions of a reaction, and global verification (`Conclusion` – `support` → `Conclusion`) generates uniform answers compared to more fluctuating final answers in AIME (`+PC2`).

F Reasoning behaviors (§6) details

Triplet pattern details. Table 10 shows the triplet patterns used for identifying behaviors in Section 6.

ReasoningFlow allows analyzing more complex structures beyond triplets, i.e., a subgraph spanning from the `Conclusion` node to retrieve *all* premises that support the conclusion, or the tree of `Planning` nodes connected via

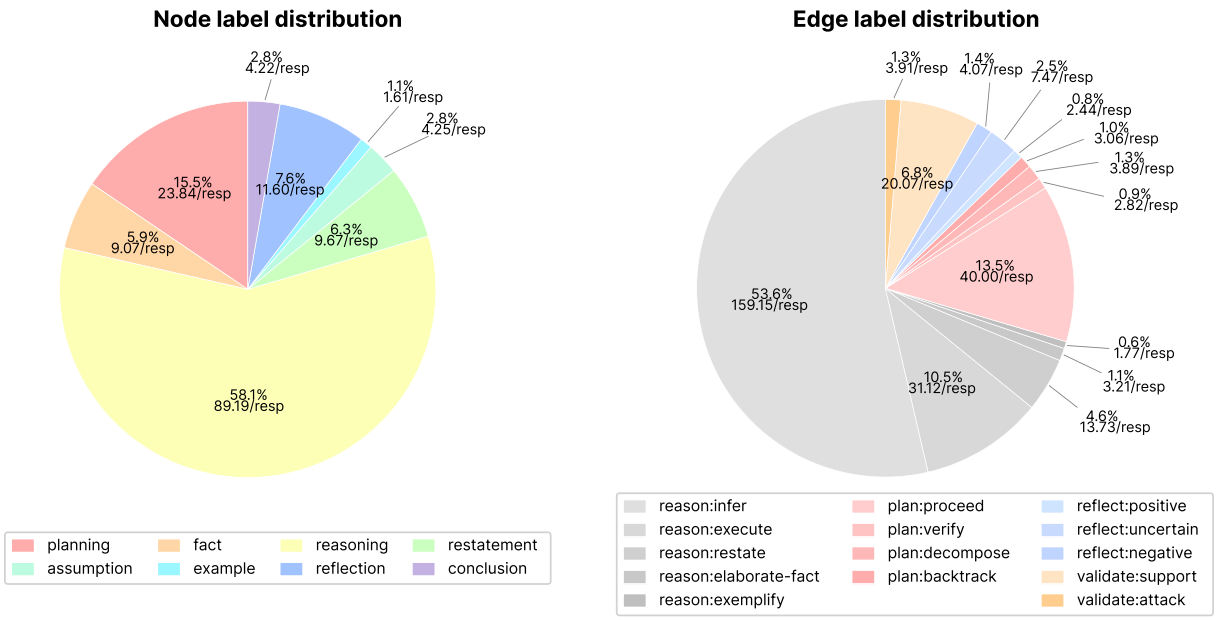


Figure 11: Node and edge label distribution in ReasoningFlow. `/resp` indicates how many times a node/edge occurs in a single LLM response.

Local Verification (§6.1)
(Reasoning) – support → Reasoning
(Reasoning) – attack → Reasoning
(Reasoning) – verify → Planning
Self-Reflection (§6.2)
(Reasoning) – positive → Reflection
(Reasoning) – uncertain → Reflection
(Reasoning) – negative → Reflection
Assumption (§6.3)
Proof-by-contradiction
Assumption – attack → Reasoning
Switch-case
Assumption – proceed → Assumption
Assumption – backtrack → Assumption

Table 10: Subgraph patterns for reasoning behavior analyzed in Section 6. For local verification and self-reflection analysis on reasoning datasets (AIME, GPQA), (Reasoning) includes Fact, Reasoning, Restatement and Conclusion.

decompose and proceed to analyze the top-down planning ability. We leave analyses of larger structures beyond triplets as future work.

Robustness against annotator models. Are the reasoning behavior analysis results robust to the selection of the annotation model? We compare the self-reflection results (Section 6.2 on DeepSeek-R1's AIME traces, with GPT-5.1 as the annotator model for nodes and edges.

Annotator	Corr.	Prop.	Error
Gemini-3	78.4	20.0	2.4
	59.6	29.8	10.7
	49.1	33.8	17.1
GPT-5.1	78.6	17.9	3.6
	73.5	19.5	7.0
	63.2	16.3	22.4

Table 11: Comparison between Gemini-3 family vs. GPT-5.1 annotations in analyzing self-reflection and step-level correctness, using DeepSeek-R1 traces from AIME dataset. Corr. and Prop. stands for Correct and Propagated, respectively.

Table 11 shows that while the actual numbers differ, the trend is preserved: positively reflected steps are most likely to be correct, and negatively reflected steps are most likely to be erroneous.

G Stepwise evaluation (§7) details

Full results. As explained in Section 7, unused errors can be attributed to excessive backtracking that leads to abandoning incorrect paths without generating a valid conclusion, and neglected errors often arise from unfaithfully ignoring orthogonal errors.

Table 12 shows the erroneous step distributions for each (model, dataset) configuration. As discussed in Section 7, QwQ and R1 both generally exhibit both a high unused rate (>80%) and a neglect rate (neglected/(neglected + faithful)>30%)

Domain	Model	Total	*Unused*	*Neglected*	*Faithful*
AIME	QwQ-32B	123	98 (79.7%)	12 (9.8%)	13 (10.6%)
AIME	DeepSeek-R1	215	192 (89.3%)	8 (3.7%)	15 (7.0%)
AIME	GPT-oss	61	39 (63.9%)	2 (3.3%)	20 (32.8%)
AIME	Qwen2.5-32B	83	28 (33.7%)	3 (3.6%)	52 (62.7%)
AIME	DeepSeek-V3	200	180 (90.0%)	0 (0.0%)	20 (10.0%)
GPQA	QwQ-32B	3398	2813 (82.8%)	203 (6.0%)	382 (11.2%)
GPQA	DeepSeek-R1	2928	2529 (86.4%)	156 (5.3%)	243 (8.3%)
GPQA	GPT-oss	1409	1060 (75.2%)	108 (7.7%)	241 (17.1%)
GPQA	Qwen2.5-32B	438	171 (39.0%)	102 (23.3%)	165 (37.7%)
GPQA	DeepSeek-V3	405	117 (28.9%)	47 (11.6%)	241 (59.5%)

Table 12: Evaluating whether the erroneous nodes are used as a premise for the final answer. Following the taxonomy of Figure 7, three columns correspond to *not reaching any conclusion* (*Unused*), *reaching a correct answer* (*Neglected*), and *reaching an incorrect answer* (*Faithful*).

for both datasets. This highlights that step-level errors seldom have an effect on final answers in these models. Therefore, solely relying on step-level validity evaluators for predicting final answer correctness of LRM traces, e.g., in Best-of-N decoding (Zou et al., 2025), is likely to underperform.

GPT-oss and Non-reasoning models (Qwen2.5-32B, DS-V3) demonstrate low unused rates in AIME, while showing significantly higher rates in GPQA. We attribute this to the multiple-choice format of GPQA; even if the intermediate step contained errors, e.g., writing an incorrect SMILES representation of a molecule, it has a much higher chance of choosing the correct answer than in AIME due to the constrained final answer space. This finding extends previous claims that LLM reasoning can be unfaithful when exposed to answer leaks or unanswerable questions (Lanham et al., 2023; Balepur et al., 2025) with quantitative evidence in reasoning trace structures.

Manual verification of unused errors. To validate the findings in Section 7, authors manually inspected 30 randomly sampled "Abandoned" nodes across LRMs and AIME/GPQA (15 each), checking whether missing ground-truth edges would have classified unused nodes otherwise. Only 10% of the cases were due to the annotation missing an edge; 90% of the unused cases were genuinely related to backtracking/local verification/assumption scopes.

In detail, the erroneous nodes were unused due to backtracking (trying a different strategy, 40.0%), local verification (explicitly reflecting on and correcting previous steps, 33.3%), and underlying assumptions being rejected (10%), highlighting the importance of understanding reasoning behaviors for evaluating the effects erroneous steps have on the model's final answer. The remaining 6.7% contained facts that are irrelevant and unnecessary for solving the problem, therefore not being connected to any subsequent nodes.

Robustness against annotator models. To show that the result is consistent across different annotator models, we compare the analysis results on DeepSeek-R1's AIME traces, with GPT-5.1 as the annotator model for nodes and edges.

The rate of unused edges is nearly identical in both annotator models, marking 89.3% and 89.7% in Gemini and GPT, respectively. Therefore, the core finding "most of the errors are not used for deriving the final answer" remains the same regardless of the annotator models. However, the rate of neglected edges changes from 3.7% to 9.2%, and faithful edges from 7.0% to 1.1%, respectively. Qualitatively, this can be partially explained by GPT-5.1's lower edge-detection recall compared to Gemini models (Table 6).

H Mechanistic Interpretability (§8) details

Full results. For Section 8, we replicate the Thought Anchor's causal dependency analyses for Qwen2.5-32B and QwQ-32B, as larger models could not be loaded on the available device. Figure 12 shows the result for both models for AIME and GPQA. The results indicate that Thought Anchor's unsupervised detection of causal dependencies is not significantly better than selecting the closest K nodes, showing the fundamental misalignment between semantic layers and causal dependencies.

Instead of causal masking, one can also analyze

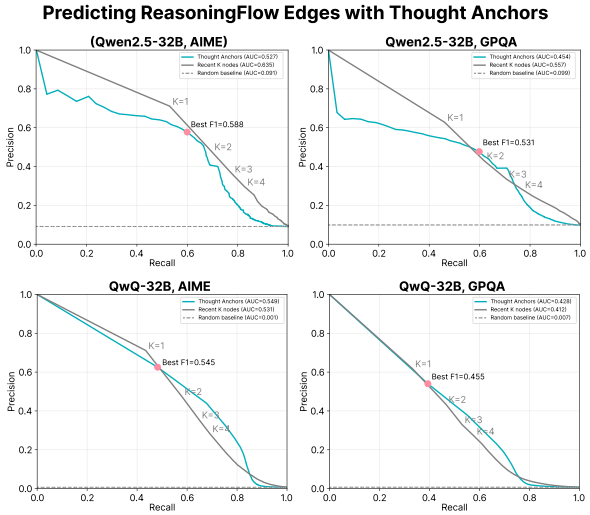


Figure 12: P-R curve of using Thought Anchors' scores to predict ReasoningFlow edges, showing that Thought Anchors are not more predictive than simply choosing K most recent nodes in all four configurations.

the *hidden state representations* with supervised or unsupervised approaches (e.g., probing, sparse auto-encoders). For instance, Zhong et al. (2026) shows that supervised probing on the difference of hidden state vectors $d_i - d_j$ can reconstruct ground-truth `infer` edges defined in synthetic reasoning benchmarks (Tafjord et al., 2021).

Manual verification of mismatched edges. To validate the findings in Section 8, we manually check 30 of the top 0.01%-scoring Thought Anchor edges that are not present in ReasoningFlow. Among these mismatched edges, only 23.3% are due to ReasoningFlow annotation errors; 76.7% are either clearly not connected or only distantly connected (path length > 1) in ReasoningFlow. This reinforces our finding that mechanistic-level and discourse-level step dependencies are different, rather than an artifact of annotation errors in ReasoningFlow.

I Prompts

This section includes the prompts used for LLM-based automatic annotators. All prompts require JSON-formatted output. For *Prompt 2: Node Classification*, we omit the definition of `Conclusion` , as they will be annotated using Prompt 3.

J License statement

The license status of all models and datasets is presented below.

- **DeepSeek-V3**: DeepSeek License v1.0
- **DeepSeek-R1**: MIT
- **Qwen2.5-32B-Instruct**: Apache 2.0
- **QwQ-32B**: Apache 2.0
- **GPT-oss-120b**: Apache 2.0
- **STILL-2**: Apache 2.0
- **NuminaMath**: Apache 2.0
- **AIME 2024**: Apache 2.0 (community re-releases)
- **GPQA Diamond**: CC-BY 4.0
- **ArgKP**: MIT

Prompt 1: Node Segmentation

Please split the following text into "reasoning units". Reasoning units are mostly sentences, but can change by whether the sentence includes complex reasoning intents. These sentences need not start with an alphanumeric character.

You need to **further split** in these cases where the sentence can be segmented into parts with different semantic roles:

- A sentence separated by a colon, where the two parts are clearly planning and its implementation. If the planning is a single word, do not separate.
  Example: "Let's now find the value of a by using the equation 3a-4=7:", "a=(7+4)/3=11/3."
- A sentence that combines an externally verifiable fact and reasoning based on the fact. *Note:* If the fact is not an absolute fact but dependent on the context/previous steps, do not split.
  Example: "Since the speed of light is 299,752,258m/s,", "the kinetic energy of a particle would be approximately 1.5M Joules."
- A sentence that links a reasoning-like phrase and a self-evaluation by punctuation (comma, colon).
  Example: "The final answer for this question is 5,", "which seems plausible to me."
- A sentence that links an assumption and its consequence.
  Example: "Assuming a=1,", "the polynomial becomes $x^2 + bx + 2b - 1 = 0$."

Note: Do not split where there are no connecting punctuations (semicolon, colon, comma, etc.)

`«input»`

Prompt 1: Node Segmentation (Cont'd)

However, do **not** split:

- Simple headers or transition words.
  Example: "1. Computing the velocity of the spacecraft"; "Case 1: x=5"; "Wait, let's verify the previous step."
- When the premise follows the conclusion in a single sentence.
  Example: "Cigarettes should not be banned, since it provides tax income that can support the federal government."; "This calculation seems off, because I know that the C-H bond length is around 1.06 to 1.10 Å."; "Since a and b are coprimes as proved above, we can ensure that there exists an inverse element of a for multiplication modulo b and vice versa."
- When a colon denotes a simple fact and its description.
  Example: "The derivative of $x\cos(x)$ is: $\cos(x) - x\sin(x)$."
- A single LaTeX equation block.
  Example: "$[x^2 - 4y = 0 \;\backslash\backslash\; y = x^2/4]$"; "`\begin{aligned}...\end{aligned}`"
- An equation followed by a step explaining the symbols used.
  Example: "The kinetic energy $E = mv^2/2$, where m is the mass and v is the velocity."

Return a JSON array of string sentences: `{"units": ["..."]}`. Preserve all punctuations and newlines exactly as given.

`«input»`

Prompt 2: Node Classification

Reasoning traces are already segmented into nodes, which are syntactically non-overlapping segments ranging from clause to one sentence (exceptionally multiline equations), and are semantically atomic. Your job is to assign labels to all nodes according to the provided description and examples below. Make sure you do not miss any nodes. Respond by a list of JSON Dicts: `{"responses": [{"node_id": "(node id)", "label": "(label)"}, ...]}`

Labels

`«node_definitions»`

Classification Rules
Reasoning traces are already segmented into nodes. Your job is to assign labels to *all* nodes. Make sure you do not miss any nodes. Respond by a list of JSON Dicts: `{"responses": [{"node_id": "(node id)", "label": "(label)"}, ...]}`

`«input»`

Prompt 3: Conclusion Node Update

Conclusion Node Update Guidelines

Conclusion. This node includes the model's final answer for the question. Conclusion nodes can directly indicate the final answer by ...
... *(examples)* ...

Annotate which of the nodes should be changed to `conclusion` nodes. Respond in JSON Dict: `{"conclusion_node_ids": ["(node id)", "(node id)", ...]}`.

`«input»`

Prompt 4: Edge Labeling

Given a destination node in a reasoning trace, find the relevant source node and edge labels according to the provided taxonomy.

- **Source nodes** should contain necessary information for the destination's reasoning process. Source nodes should cover **all** crucial antecedents, logical premises, background knowledge, and core plans/ideas of the destination step.
- Note that in most cases, with most common exceptions in `reason:restate` and `validate:support/refute`, source nodes are placed closely within the destination node.
- Each edge label has information about "possible source labels". The selected source node and edge labels must match.
- For certain types of nodes (e.g., reflection, fact), there might be no source node. If no source node satisfies the definitions below, you can return an empty list.

Edge label definitions
`«edge_definitions»`

Respond in JSON format by listing all edges connecting to the "destination step":

```
[{ "source_node_id": "(node id)",
"label": "(label)" }]
```

Few-shot examples
`«few_shot_examples»`

Actual input
`«input»`